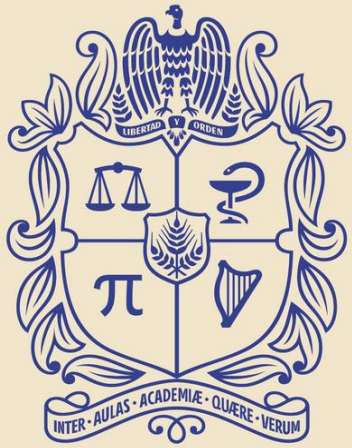


UNIVERSIDAD
NACIONAL
DE COLOMBIA

INTRODUCTION

Daniel Felipe Rodríguez Ramírez, MIG

dafrodriguezram@unal.edu.co

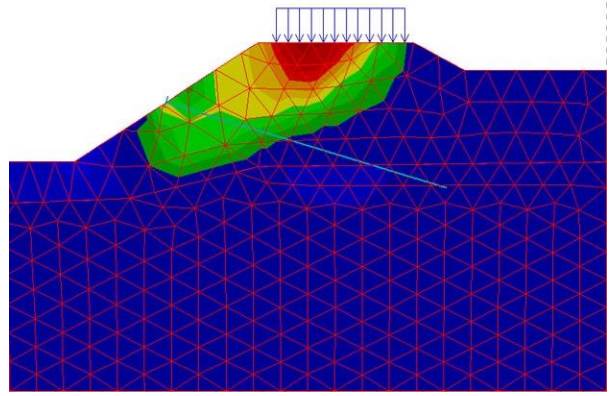
CIUDAD UNIVERSITARIA, BOGOTÁ D.C.

Content

1. Syllabus

Numerical Methods for Engineering

This course provides an understanding of numerical methods applied to the broad field of engineering. Students will learn how to formulate mathematical models, handle numerical errors, solve equations (linear and nonlinear), perform numerical integration and differentiation, and tackle differential equations (ordinary and partial) using computer-based methods and specialized software.



Numerical modeling of a slope

Learning Outcomes

1. **Identify and formulate** engineering problems that can be approached with numerical methods, selecting appropriate models and computational tools.

2. **Apply and analyze** fundamental numerical algorithms (e.g., for root-finding, linear systems, optimization) to solve complex engineering tasks.

3. **Evaluate the accuracy and stability** of numerical results, recognizing common sources of error and proposing strategies to minimize their effects.

4. **Implement and validate** numerical solutions using modern software (MATLAB, Python, or equivalent), interpreting outcomes in the context of specific engineering disciplines.

5. **Communicate** technical findings effectively, demonstrating how numerical results inform design decisions and problem-solving in engineering contexts.

Course Outline

Part 1: Introduction - Modeling, Computers, and Error Analysis

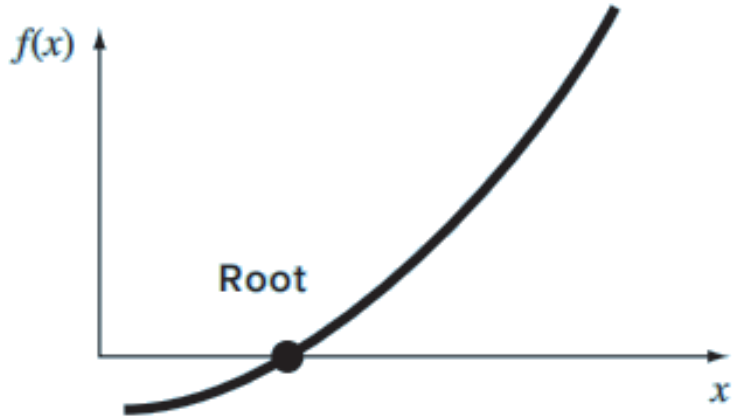
- Mathematical Modeling and Engineering Problem-Solving
- Programming and Software
- Approximations and Round-Off Errors
- Truncation Errors and Taylor Series

Key Objective (Part 1): Gain a solid foundation in mathematical modeling, understand the significance of computers in engineering calculations, and learn how different types of error affect numerical solutions.

Course Outline

Part 2: Roots of Equations

- Bracketing Methods
- Open Methods
- Roots of Polynomials
- Case Studies: Roots of Equations

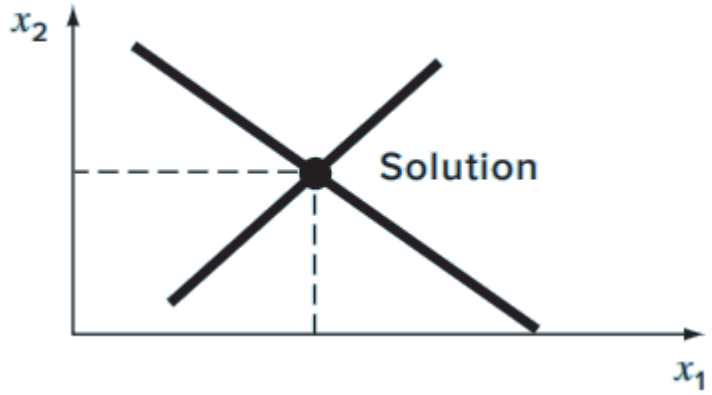


Key Objective (Part 2): Master numerical methods for solving nonlinear equations (single and multiple roots, polynomial roots), with emphasis on practical engineering scenarios.

Course Outline

Part 3: Linear Algebraic Equations

- Gauss Elimination
- LU Decomposition and Matrix Inversion
- Special Matrices and Gauss-Seidel Method
- Case Studies: Linear Algebraic Equations

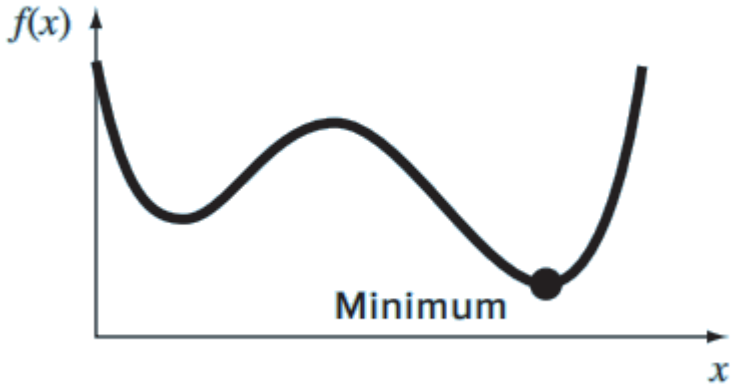


Key Objective (Part 3): Develop proficiency in solving large systems of linear equations using direct and iterative methods, considering error sources and numerical stability.

Course Outline

Part 4: Optimization

- One-Dimensional Unconstrained Optimization
- Multidimensional Unconstrained Optimization
- Constrained Optimization
- Case Studies: Optimization

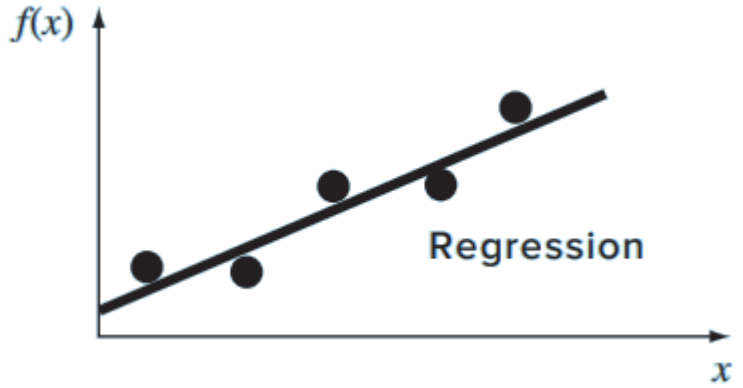


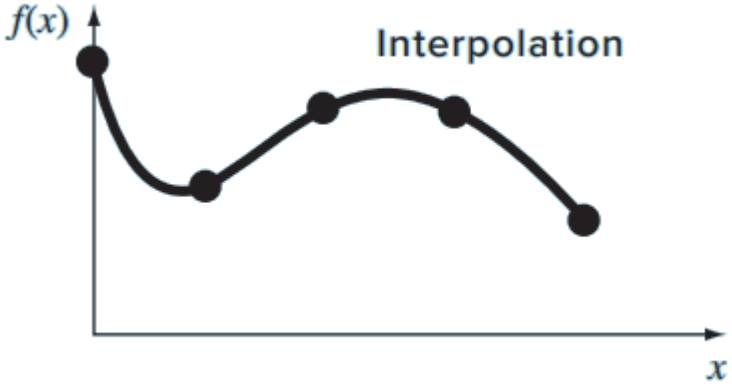
Key Objective (Part 4): Acquire knowledge of optimization strategies (with or without constraints) and learn to apply them to engineering design and analysis.

Course Outline

Part 5: Curve Fitting

- Least-Squares Regression
- Interpolation
- Fourier Approximation
- Case Studies: Curve Fitting



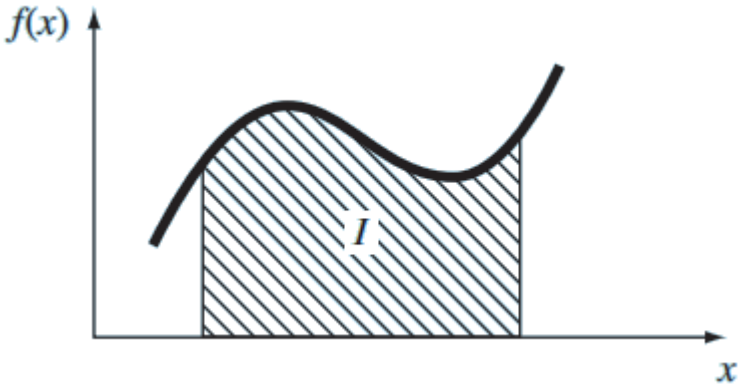


Key Objective (Part 5): Learn numerical methods for fitting data and functions (regression, interpolation, and Fourier techniques) to support engineering applications that require signal analysis and function approximation.

Course Outline

Part 6: Numerical Differentiation and Integration

- Newton-Cotes Integration Formulas
- Integration of Equations
- Numerical Differentiation
- Case Studies: Numerical Integration and Differentiation

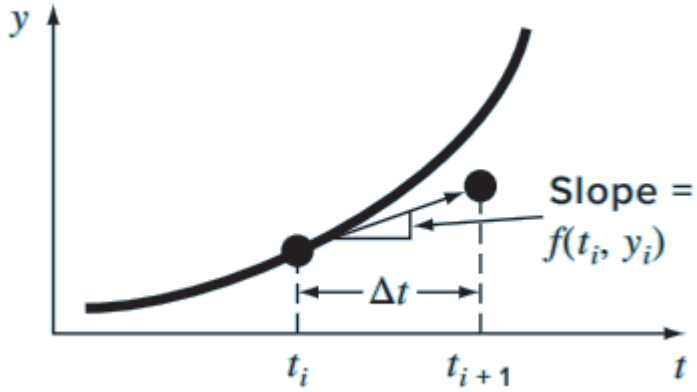


Key Objective (Part 6): Apply numerical integration and differentiation methods to solve real engineering problems, understand error bounds, and handle complex data sets.

Course Outline

Part 7: Ordinary Differential Equations (ODEs)

- Runge-Kutta Methods
- Stiffness and Multistep Methods
- Boundary-Value and Eigenvalue Problems
- Case Studies: Ordinary Differential Equations



Key Objective (Part 7): Develop the ability to solve ODEs (initial and boundary-value problems) arising in engineering contexts, including stiff and eigenvalue-related problems.

Expected Weekly Distribution

Weeks 1–2: Part 1 – Modeling and Error Analysis

Weeks 3–4: Part 2 – Roots of Equations

Weeks 5–6: Part 3 – Linear Algebraic Equations

Weeks 7–8: Part 4 – Optimization

Weeks 9–10: Part 5 – Curve Fitting

Weeks 11–12: Part 6 – Numerical Integration and Differentiation

Weeks 13–14: Part 7 – Ordinary Differential Equations

Assessment Methods

- **Workshops and miniprojects**:

Problems solved using software tools (MATLAB, Python, etc.). (20%)

- **Midterm Exams:**

3 throughout the term. (20% each one)

- **Final Exam or Final Project**:

Integrative application of methods studied to a real-world engineering problem. (20%)

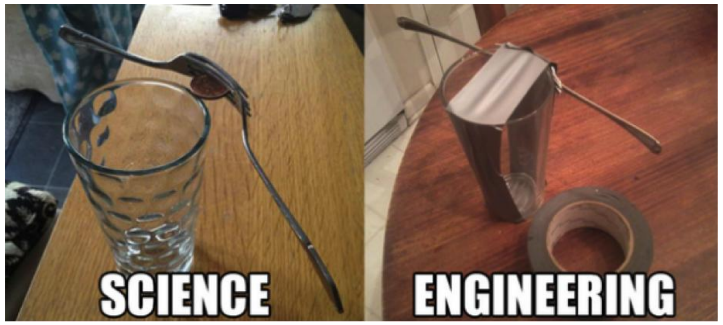
2. Introduction

Definition of Numerical Methods

- Numerical methods are techniques used to **reformulate** mathematical problems so they can be solved using arithmetic operations.

- They typically involve large numbers of repetitive arithmetic calculations, which has become practical and efficient with the advent of digital computers.

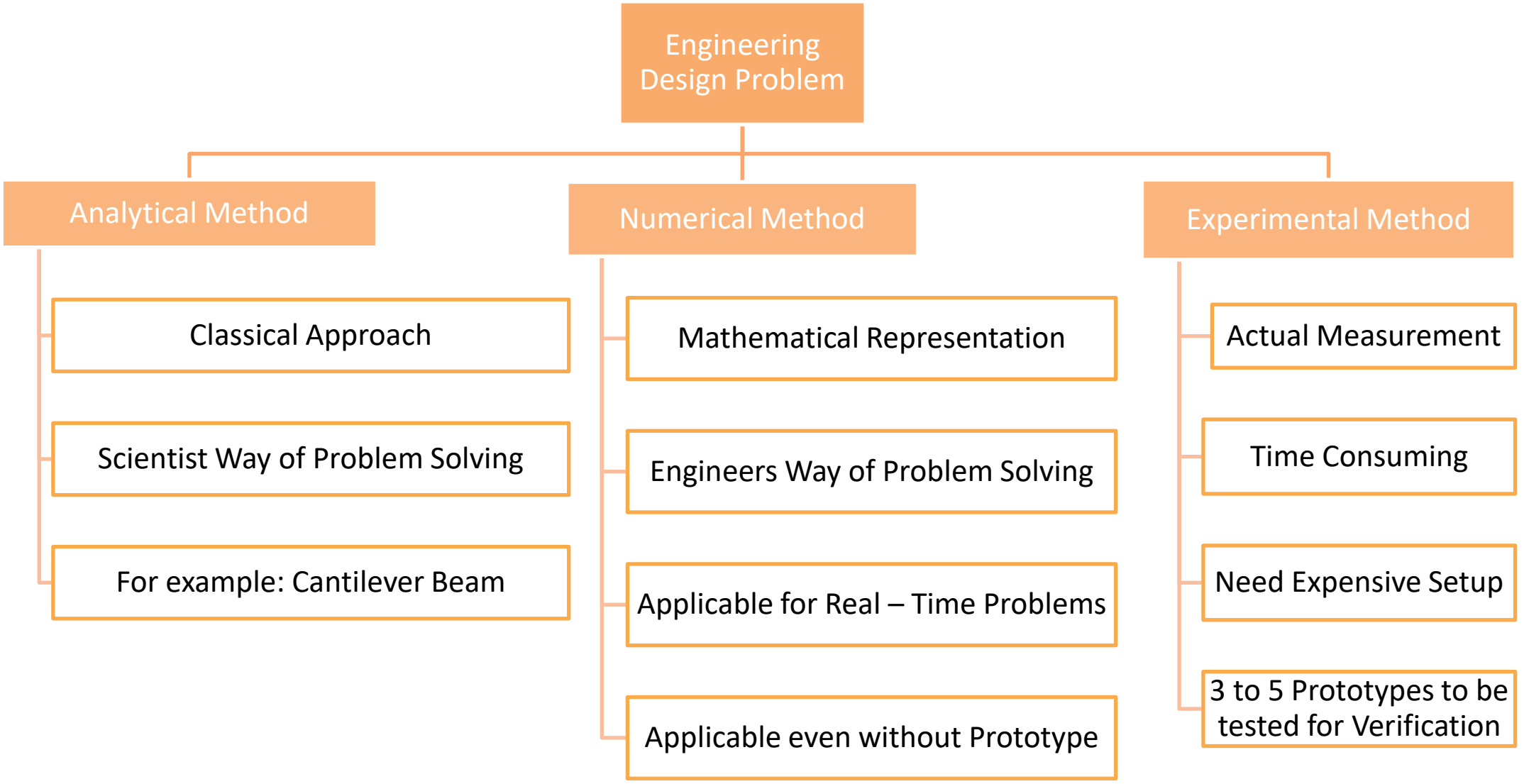
Engineers or scientists?
Source: The Big Bang Theory



Definition of Numerical Methods

"Perfect is the Enemy of Good."
- Voltaire

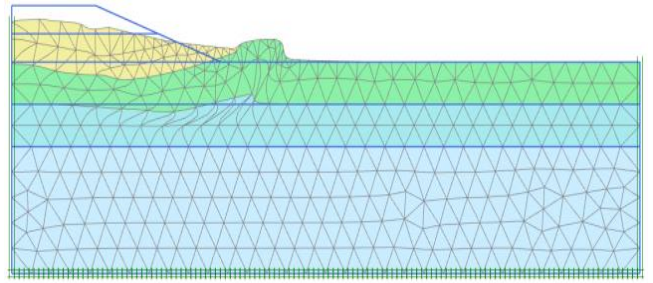
"Science is about knowing; engineering is about doing." - Henry Petroski


Engineering Design Problem
Analytical Method
Classical Approach
Scientist Way of Problem Solving
For example: Cantilever Beam
Numerical Method
Mathematical Representation
Engineers Way of Problem Solving
Applicable for Real – Time Problems
Applicable even without Prototype
Experimental Method
Actual Measurement
Time Consuming
Need Expensive Setup
3 to 5 Prototypes to be tested for Verification

Limitations of Analytical (Exact) Solutions

- Analytical solutions offer valuable insight but can only be derived for a limited class of problems (e.g., those approximated by linear models or with simple geometry).

- Real engineering problems often involve complex shapes and nonlinear processes, making purely analytical approaches of limited practical value.

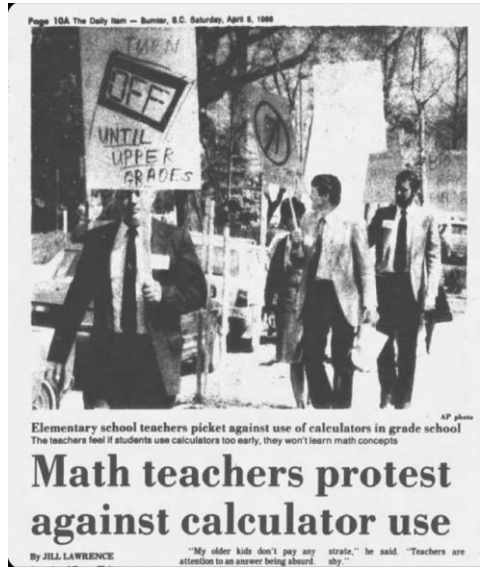


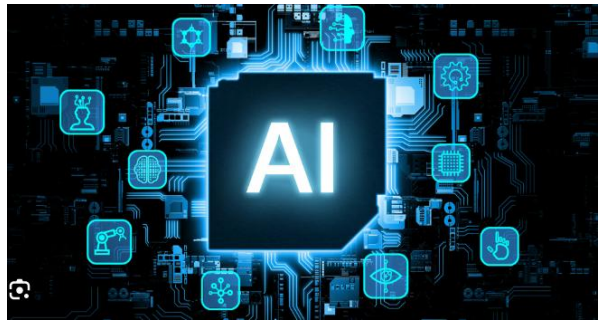
Slope analysis

Use of Calculators and Slide Rules

- Before widespread computer use, manual methods (calculators, slide rules) were employed to implement numerical techniques.

- Although they can theoretically solve complex problems, manual calculations are tedious, error-prone, and slow.



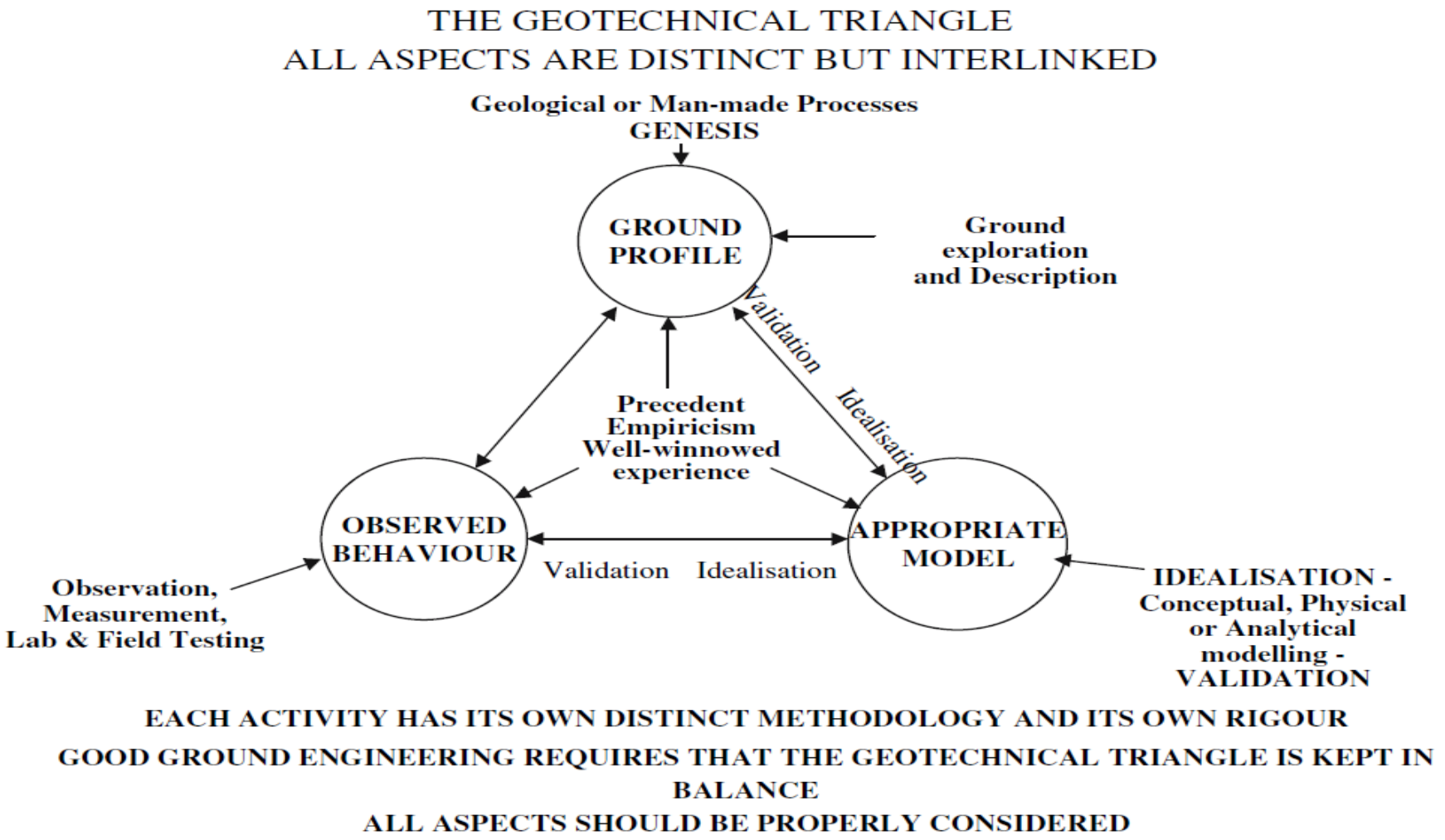


Focus on Technique Rather than Problem Definition

- Significant energy was spent on the solution technique itself, rather than on **problem formulation and interpretation**, due to the labor-intensive nature of precomputer methods.

- This diverted attention from the **creative and interpretative aspects** of problem solving.

Focus on Technique Rather than Problem Definition



Taken from: Burland (2007) Terzaghi: back to the future. Bull Eng Geol Environ. 66: 29-33

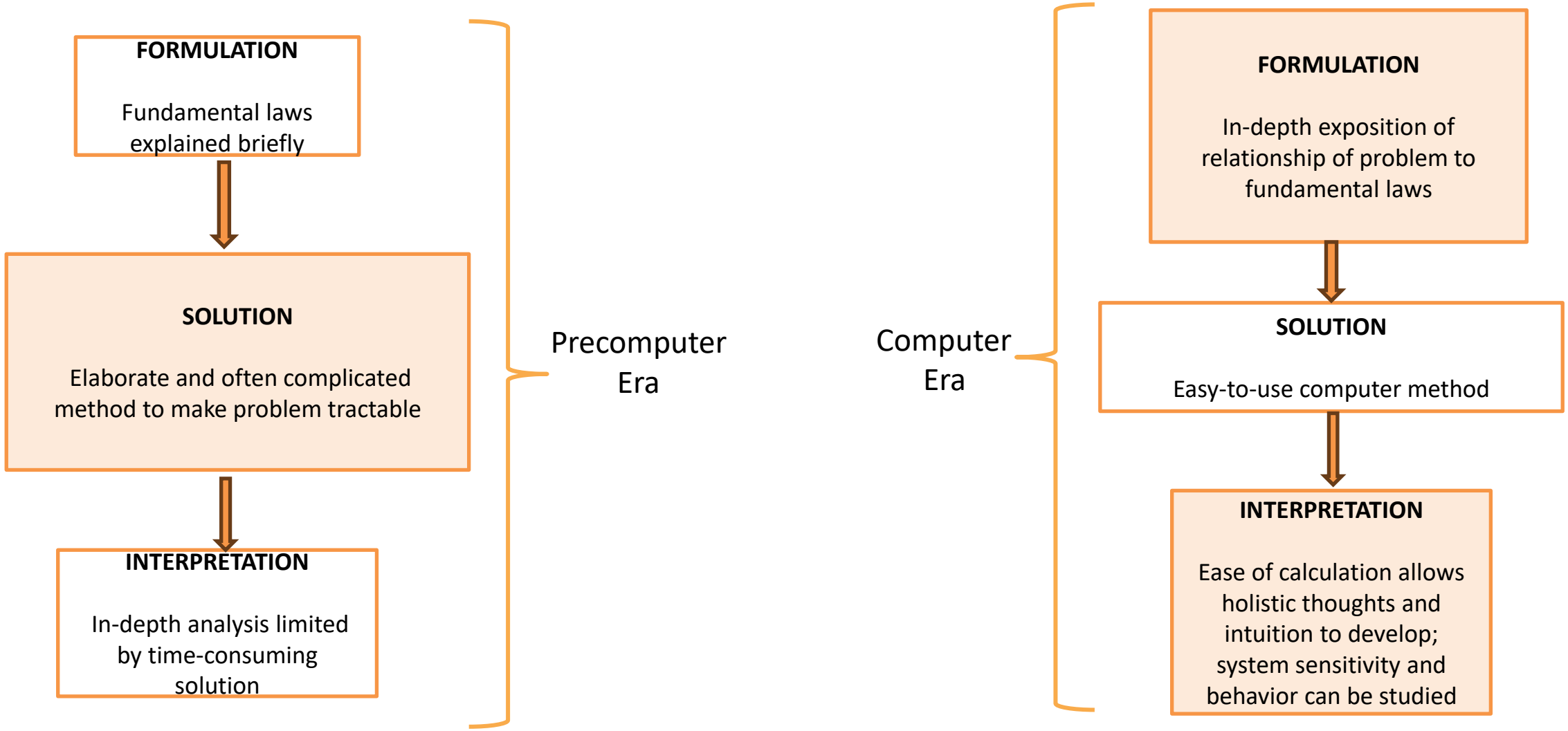
Impact of Computers and Modern Numerical Methods

- The availability of fast computing power **eliminates the need for simplifying assumptions** or time-intensive manual tasks.

- Engineers can now focus more on **defining problems comprehensively, interpreting solutions, and incorporating holistic system awareness**.

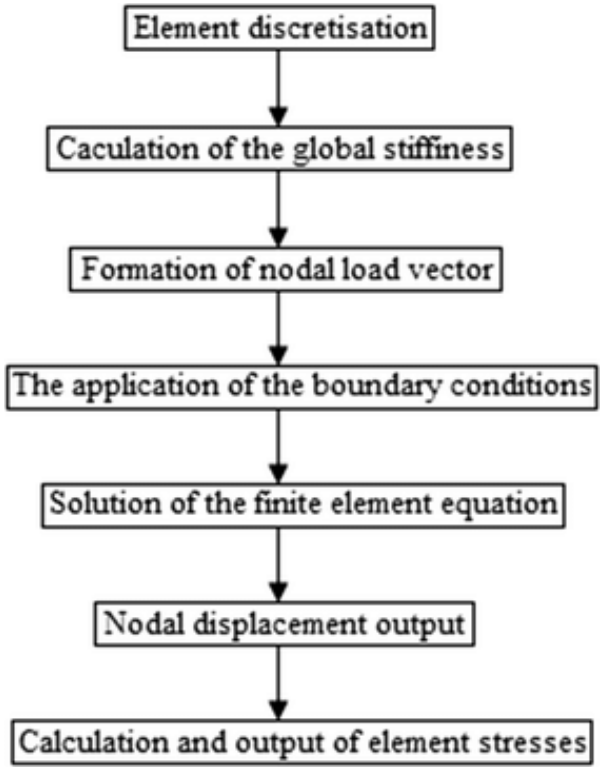
- Analytical methods remain valuable for insight, but **numerical techniques greatly expand problem-solving capabilities**.

Phases of engineering problem solving



Overview of Numerical Approaches

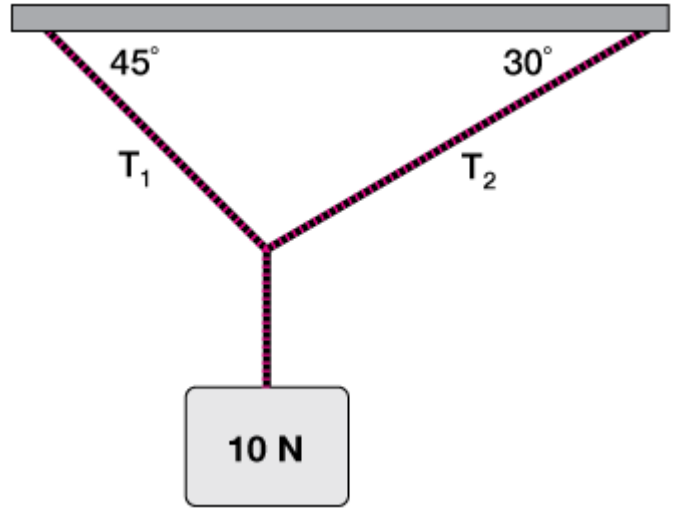
- Numerical methods involve transforming mathematical problems into sequences of arithmetic calculations.
- They allow for intensive and repetitive computations, made practical by digital computers.



General steps in FEM

When Exact Math Fails to Deliver

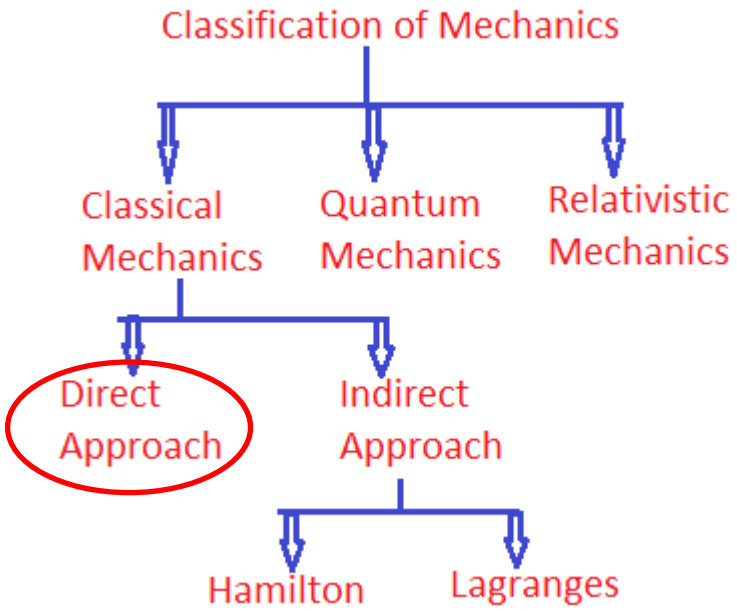
- Analytical (or exact) solutions are useful for insight, yet they are only feasible for simpler, often linear problems.
- Engineering challenges frequently feature complex geometries and nonlinearities, limiting pure analytical methods.



3. Mathematical Modeling and Engineering Problem Solving

Definition of Mathematical Modeling

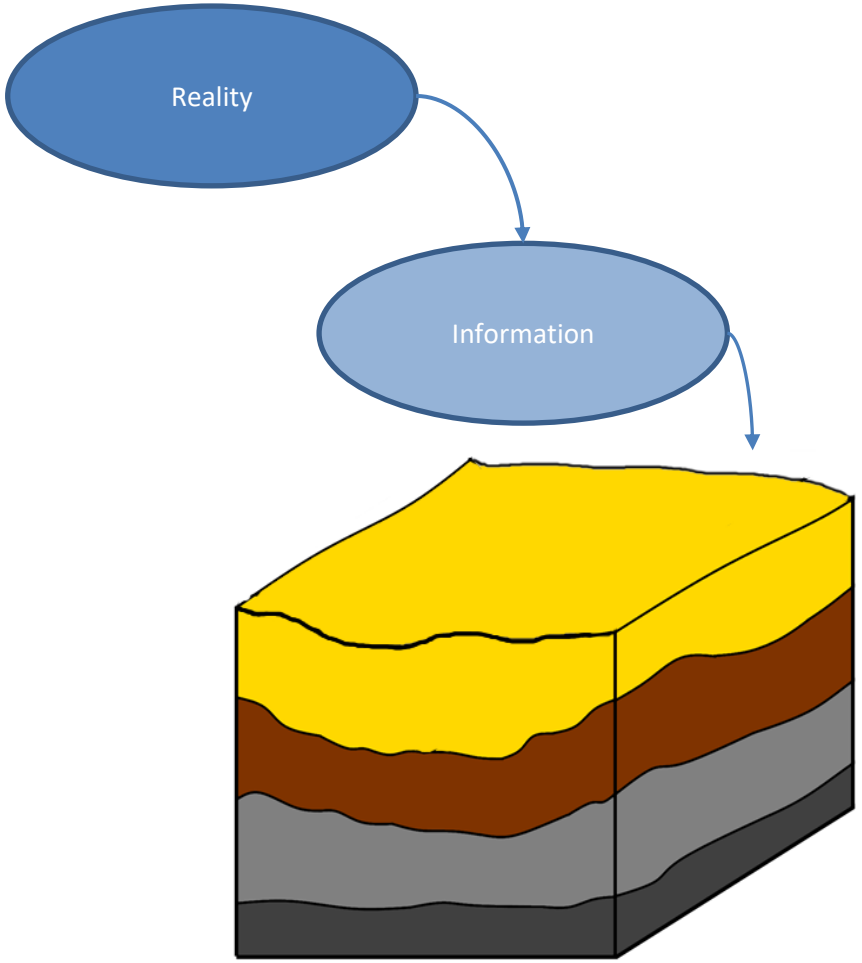
- Mathematical modeling involves representing a real-world system or process using equations and logical frameworks.
- It simplifies complex phenomena by focusing on key factors and ignoring minor or negligible influences that do not significantly affect overall behavior.



Empirical Observation and Theoretical Analysis

- Engineering problem solving relies on two primary sources: empirical data (experiments and measurements) and theoretical principles (fundamental laws like Newton's laws).

- A good understanding of the physical system is necessary to build an appropriate model and to interpret results meaningfully.

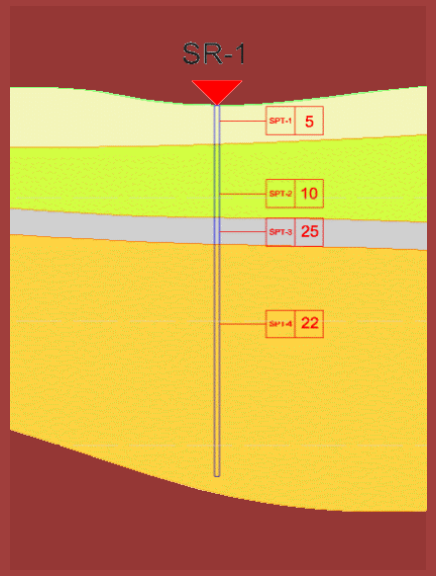


Empirical Observation and Theoretical Analysis

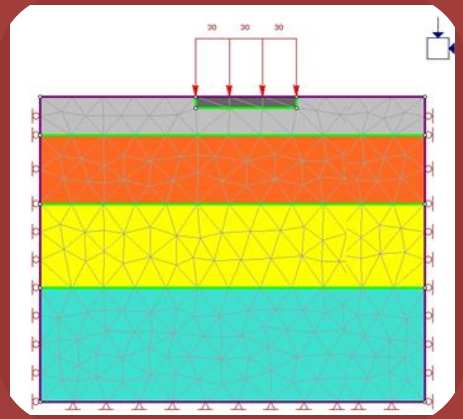
Field

Laboratory

Model



Analysis



General Form of a Model

Models often take the form:

$$Dependent\ variable = f(independent\ variables, parameters, forcing\ functions)$$

- **Dependent Variable:** The quantity whose behavior we want to predict (e.g., velocity).
- **Independent Variables:** Dimensions along which the system evolves (e.g., time, space).
- **Parameters:** System-specific properties that remain constant in a particular analysis (e.g., mass, drag coefficient).
- **Forcing Functions:** External inputs or influences (e.g., gravity).

Falling Parachutist Example (Linear Drag)

Forces Involved:

1. **Downward (Gravity)**: $F_D = mg$
2. **Upward (Drag)**: $F_U = -cv$

where c is a proportionality constant called the drag coefficient (kg/s).

And, v is the velocity.

Falling Parachutist Example (Linear Drag)

Differential Equation:

$$\frac{dv}{dt} = \frac{F}{m}$$

$$\frac{dv}{dt} = \frac{mg - cv}{m} \quad \Rightarrow \quad \frac{dv}{dt} = g - \frac{c}{m}v$$

Analytical Solution:

Forcing function — Parameter — Independient variable

$$v(t) = \frac{gm}{c}\left(1 - e^{-\left(\frac{c}{m}\right)t}\right)$$

Dependent variable — Parameter

Analytical, or exact, solution

Falling Parachutist Example (Linear Drag)

Terminal Velocity

After a long time, the exponential term in the analytical solution decays to zero, so velocity becomes constant at:

$$v_{terminal} = \frac{gm}{c}$$

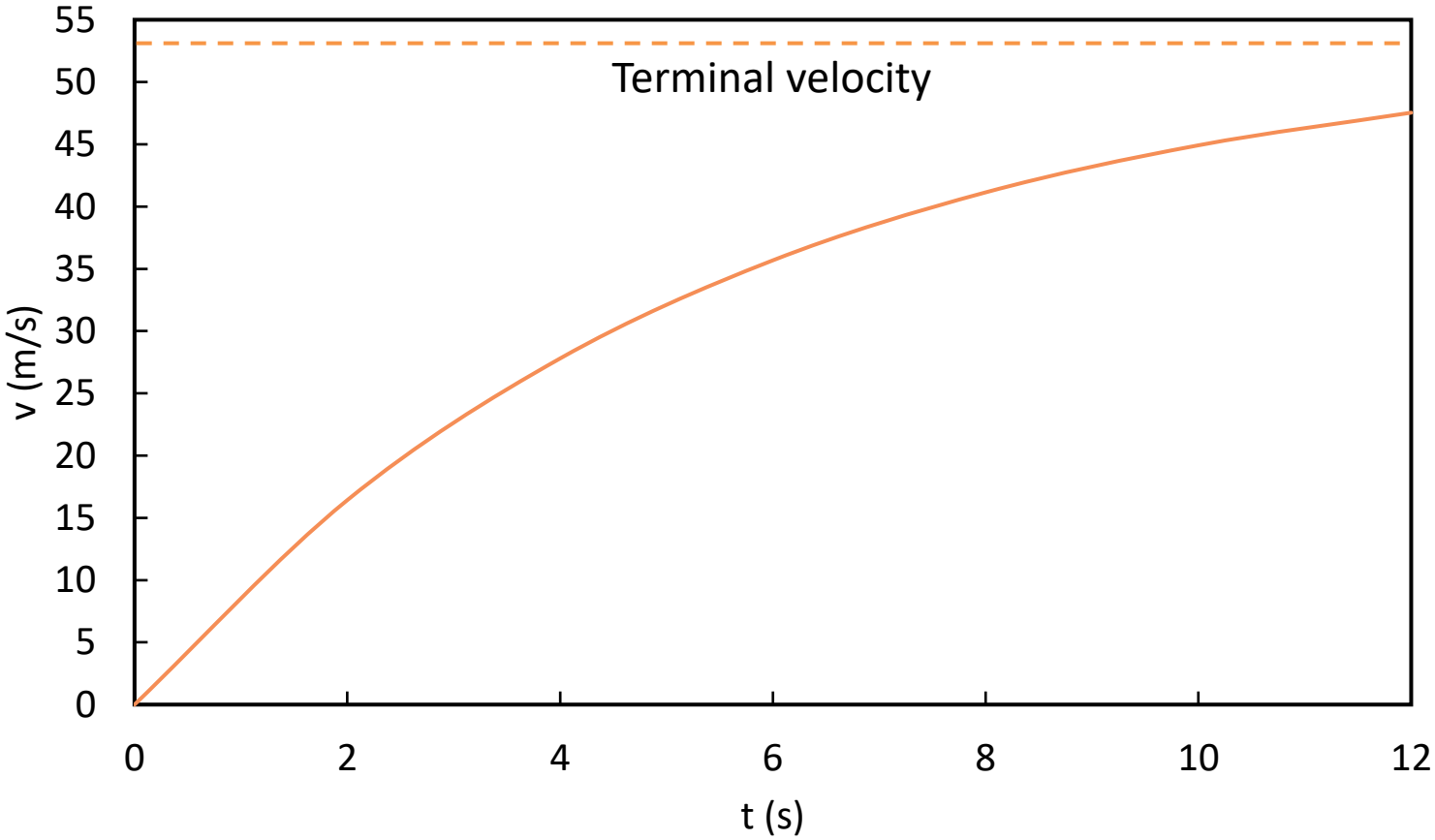
At terminal velocity, the gravitational force and the drag force balance each other, resulting in zero net force and zero acceleration.

Example 1

To compute velocity prior to opening the chute.

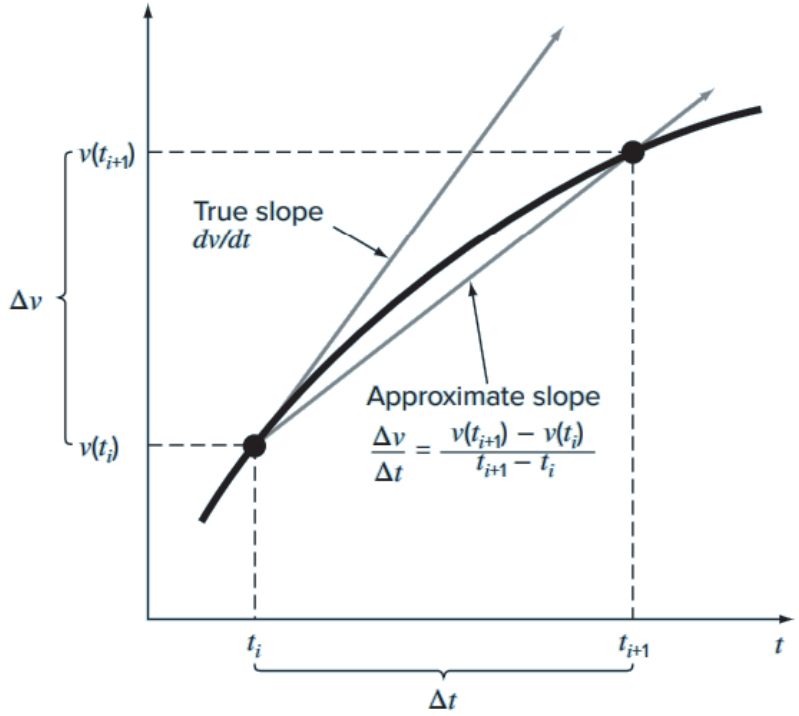
$m = 68.1\ kg$
$c = 12.5\ kg/s$

Example 1


Terminal velocity
v (m/s)
t (s)
0
5
10
15
20
25
30
35
40
45
50
55
0
2
4
6
8
10
12

Numerical Methods (Euler's Method)

- Many real-world problems cannot be solved with simple algebra or even standard calculus methods.
- Numerical methods approximate solutions by converting continuous equations into step-by-step algebraic updates.
- Euler's Method for the falling parachutist approximates the derivative $\frac{dv}{dt}$ via **finite differences:**



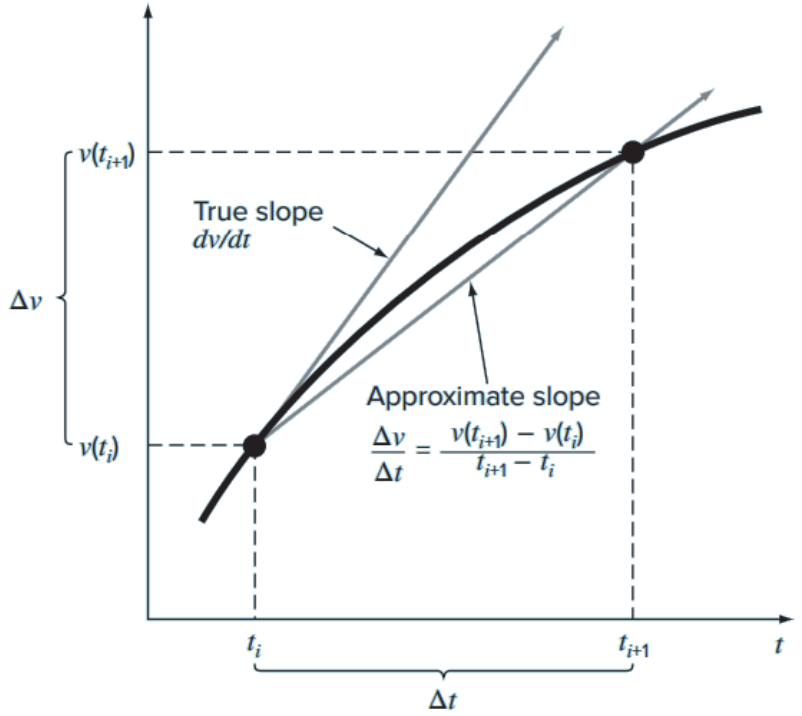
$$\frac{dv}{dt} \cong \frac{\Delta v}{\Delta t} = \frac{v(t_{i+1}) - v(t_i)}{t_{i+1} - t_i}$$

$$\frac{dv}{dt} = \lim_{\Delta t \to 0} \frac{\Delta v}{\Delta t}$$

Numerical Methods (Euler's Method)

$$\frac{v(t_{i+1}) - v(t_i)}{t_{i+1} - t_i} = g - \frac{c}{m}v(t_i)$$

$$v(t_{i+1}) = v(t_i) + \left[g - \frac{c}{m}v(t_i)\right](t_{i+1} - t_i)$$



What happens if Δt is very small?

Accuracy vs. Computational Effort

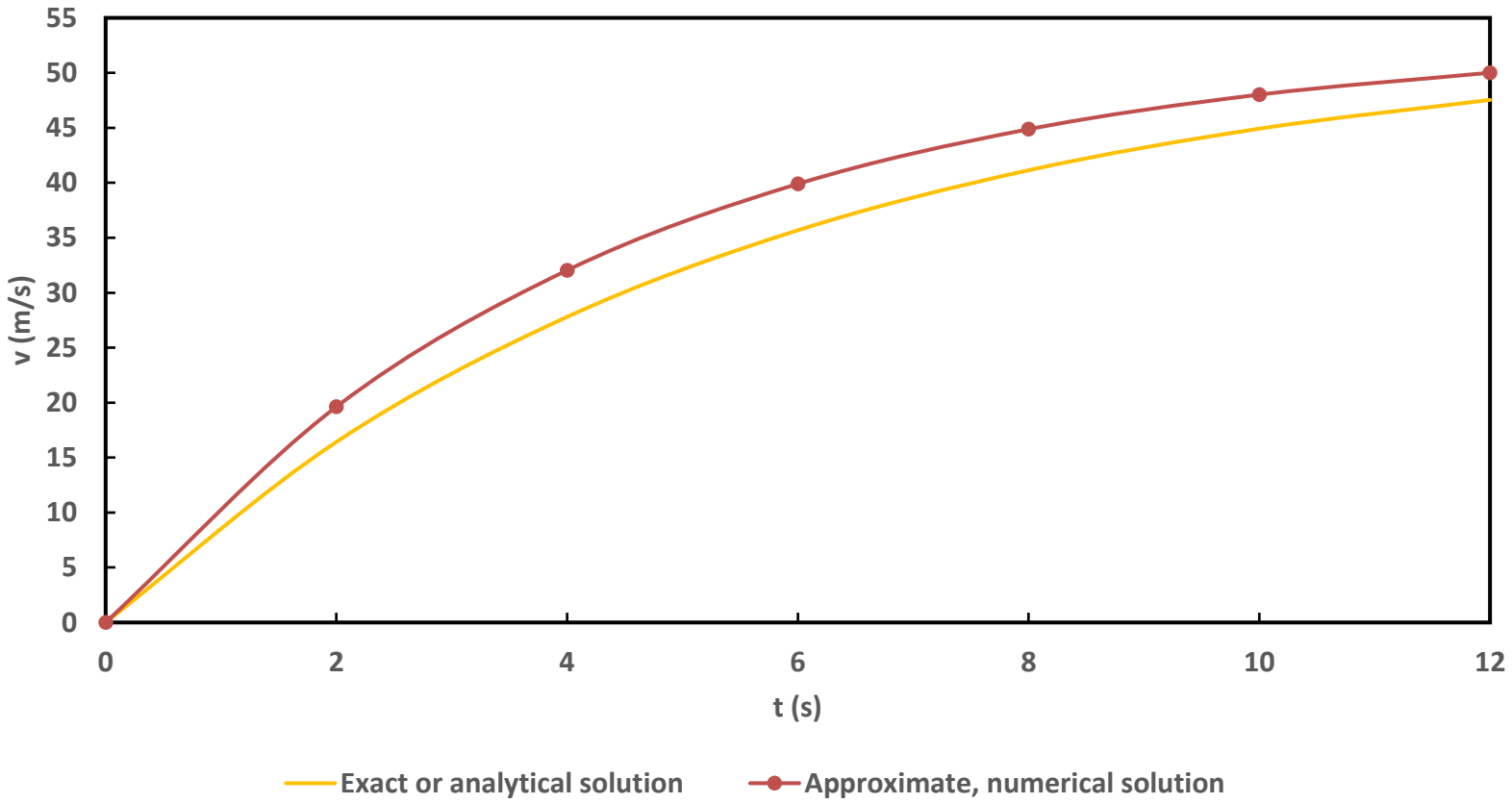
- Step Size (Δt): Smaller Δt yields more accurate results but increases the number of computations.
- Numerical solutions form straight-line approximations between intervals, thus **the total error depends on step size**.
- Modern computers allow extremely fine step sizes without prohibitive manual effort.

Example 2

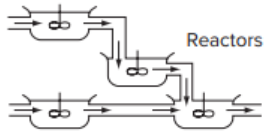
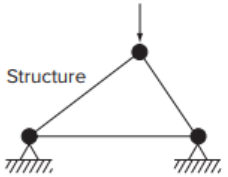
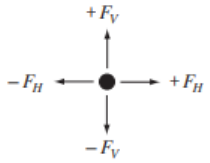
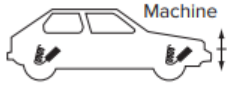
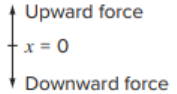
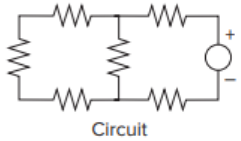
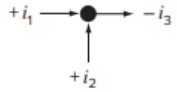
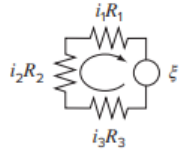
Perform the same computation as in Example 1 but use Euler's Method. Employ a step size of $\Delta t = 2\ s$

$m = 68.1\ kg$
$\mathrm{c} = 12.5\ kg/\mathrm{s}$

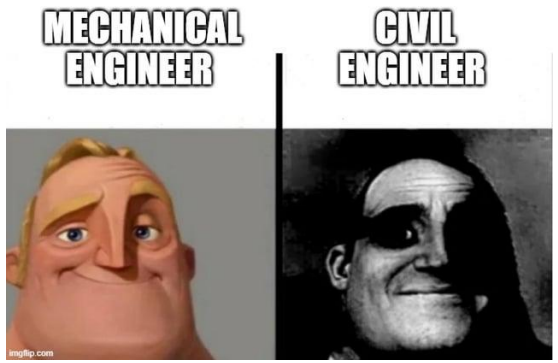
Example 2


55
50
45
40
35
30
25
20
15
10
5
0
v (m/s)
0
2
4
6
8
10
12
t (s)
Exact or analytical solution
Approximate, numerical solution

Major Conservation Laws in Engineering and its applications

Field	Device	Organizing Principle	Mathematical Expression
Chemical engineering		Conservation of mass	Mass balance:  Over a unit of time period Δmass = inputs – outputs
Civil engineering		Conservation of momentum	Force balance:  At each node Σ horizontal forces (F_H) = 0 Σ vertical forces (F_V) = 0
Mechanical engineering		Conservation of momentum	Force balance:  $m\frac{d^2x}{dt^2}$ = downward force – upward force
Electrical engineering		Conservation of charge	Current balance:  For each node Σ current (i) = 0
		Conservation of energy	Voltage balance:  Around each loop Σ emf's – Σ voltage drops for resistors = 0 Σ ξ – Σ iR = 0

It's moving



4. Programing

Types of Software Users

Standard Users:

- Use software "as is" with built-in functions (e.g., Excel, MATLAB) to solve typical problems like linear systems or plotting x-y data.

Power Users:

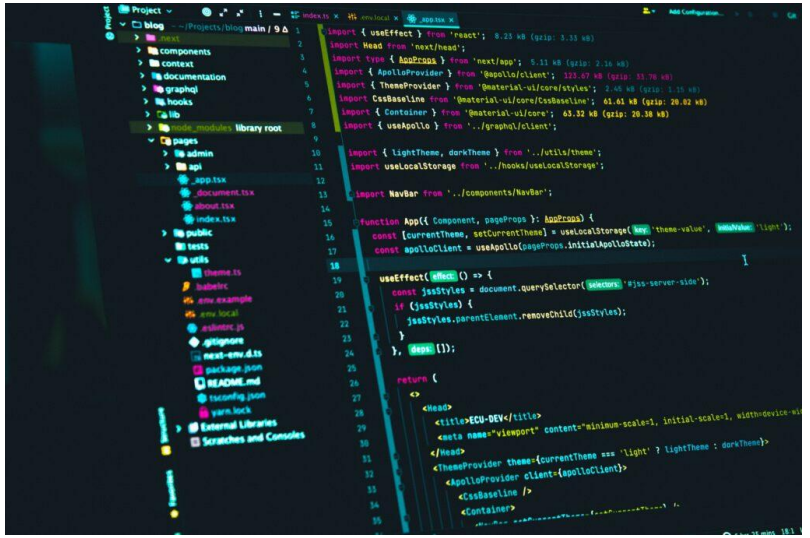
- Extend software functionality by writing custom code, such as VBA macros in Excel, M-files in MATLAB, or scripts in Python

Motivation for Programming

- Encourages engineers not to be constrained by the standard features of software packages

- Two main alternatives when facing problems outside a tool's standard capabilities:
  - Find another package with the needed features
  - Write custom programs or scripts (in languages like Python, Fortran 90, C, MATLAB, or VBA) to extend the tool's functionality

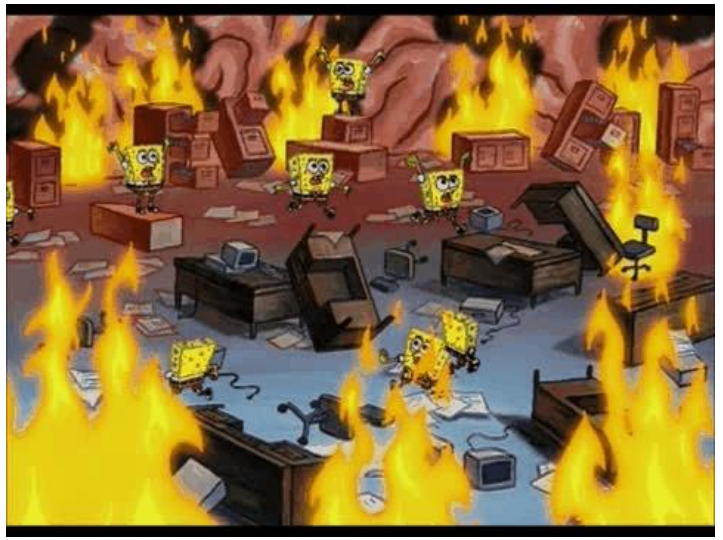
- Highlights the transferable nature of programming skills across different languages and environments



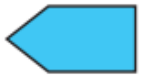
Structured Programming

Importance of Code Structure

- Well-organized and clearly structured code is easier to debug, test, share, and maintain
- Structured programming minimizes errors and improves the overall efficiency of program development



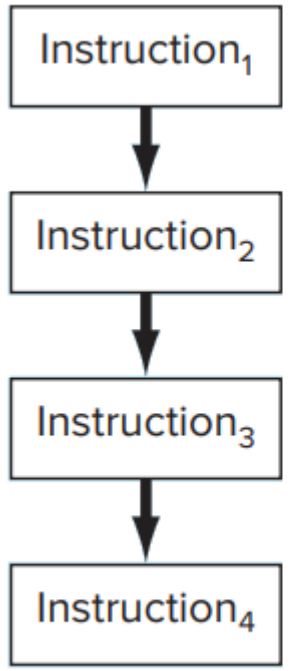
Structured Programming

SYMBOL	NAME	FUNCTION
	Terminal	Represents the beginning or end of a program.
	Flowlines	Represent the flow of logic. The humps on the horizontal arrow indicate that it passes over and does not connect with the vertical flowlines.
	Process	Represents calculations or data manipulations.
	Input/output	Represents inputs or outputs of data and information.
	Decision	Represents a comparison, question, or decision that determines alternative paths to be followed.
	Junction	Represents the confluence of flowlines.
	Off-page connector	Represents a break that is continued on another page.
	Count-controlled loop	Used for loops that repeat a prespecified number of iterations.

Fundamental Control Structures

Sequence Structure

- Instructions are executed in a linear, step-by-step manner
- Illustrated using both **flowcharts** (graphical symbols) and **pseudocode** (language-agnostic step descriptions)



Flowchart

```
Instruction1
Instruction2
Instruction3
Instruction4
```

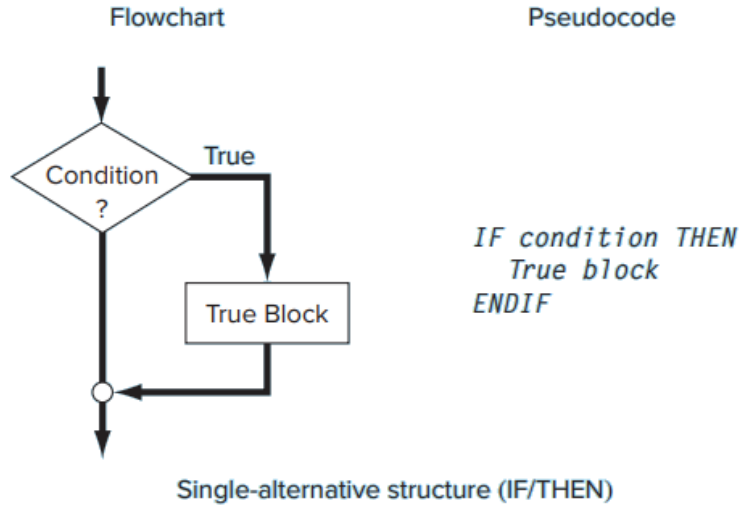
Pseudocode

Fundamental Control Structures

Selection Structures (Decision Making)

Single-Alternative (IF/THEN):

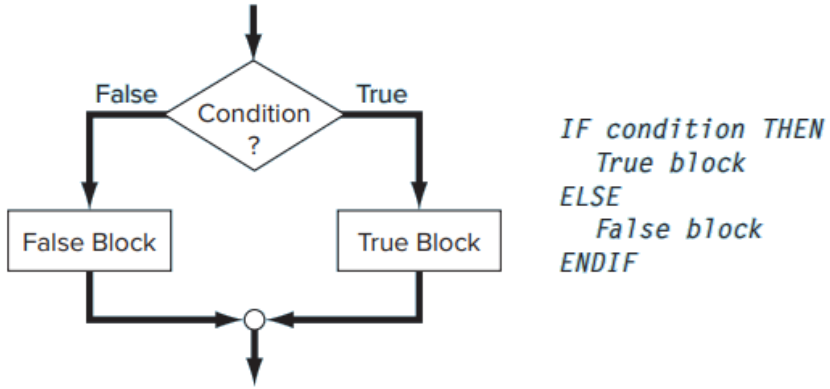
- Executes a block only when a condition is true



Single-alternative structure (IF/THEN)

Double-Alternative (IF/THEN/ELSE):

- Provides two branches to execute code based on whether a condition is true or false



Double-alternative structure (IF/THEN/ELSE)

Fundamental Control Structures

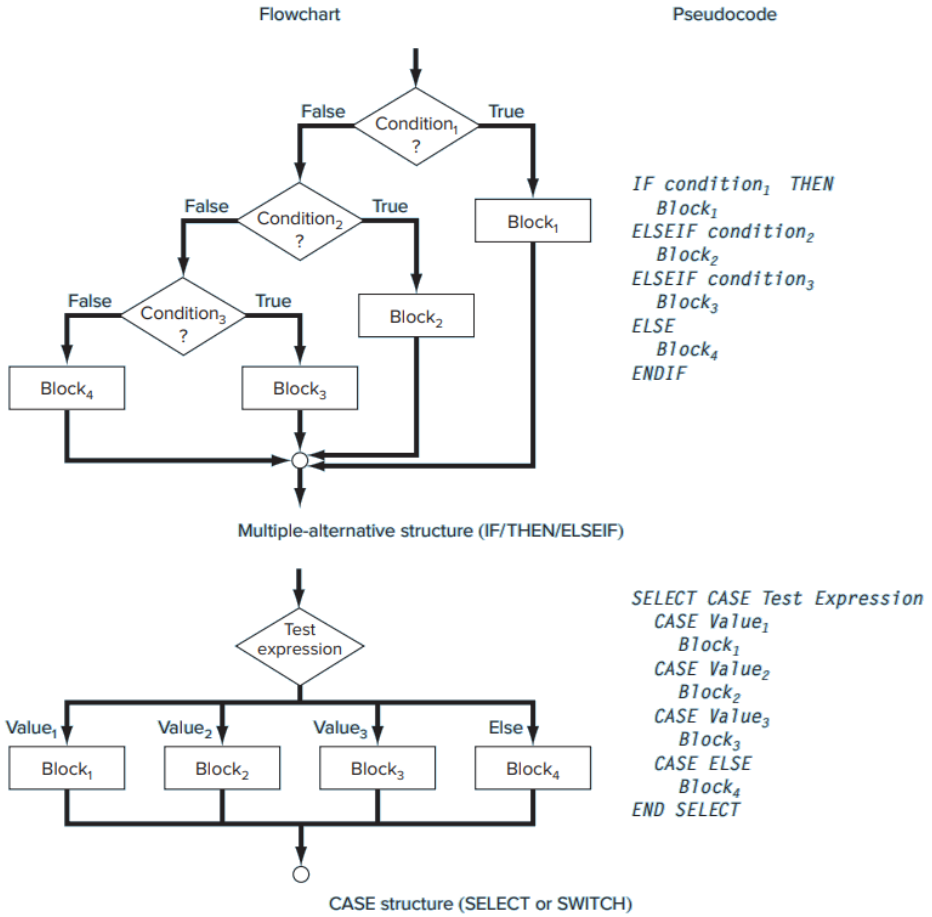
Selection Structures (Decision Making)

Multiple-Alternative (IF/THEN/ELSEIF):

- Chains multiple conditions so that the first true condition determines the executed block

CASE Structure:

- Uses a single expression's value to select between several alternatives

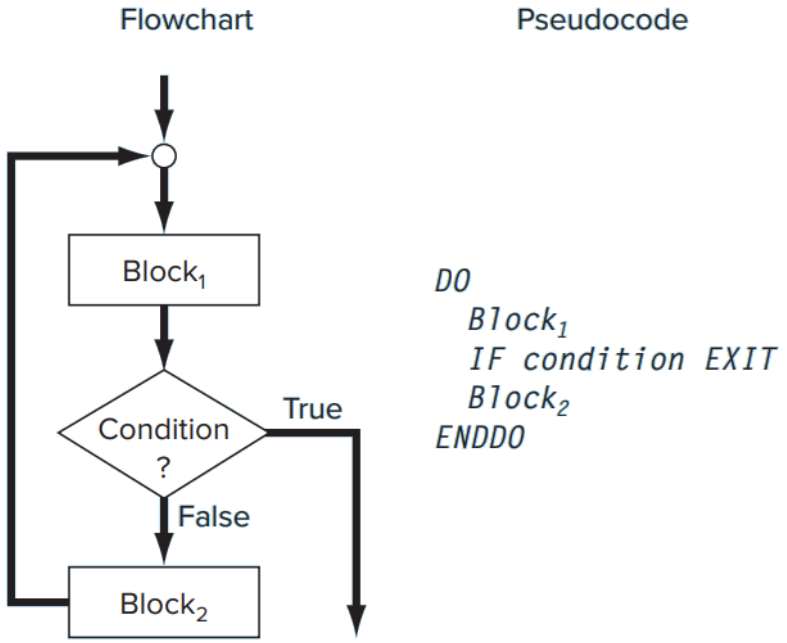


Fundamental Control Structures

Repetition Structures (Loops)

Decision Loops (DOEXIT or Break Loops):

- Continuously execute a block until a specific condition is met (commonly implemented with a pretest or posttest loop)
- In languages like MATLAB or Python, a WHILE loop (with a break statement) can simulate this behavior

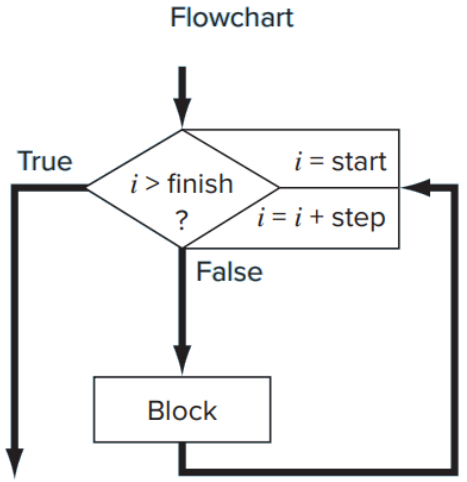


Fundamental Control Structures

Repetition Structures (Loops)

Count-Controlled Loops (DOFOR Loops):

- Repeat a block a set number of times using an index variable
- In Python, such loops are typically implemented using the “for” statement with a range (e.g., for i in range(start, finish))



Example 3

Solve $ax^2 + bx + c = 0$ using the quadratic formula.

Develop an algorithm that does the following:

1. Prompts the user for the coefficients, a, b, and c
2. Implements the quadratic formula, guarding against all eventualities (for example, avoiding division by zero and allowing for complex roots).
3. Displays the solution, that is, the values for x.
4. Allows the user the option to return to step 1 and repeat the process.

Solution Example 3 in MATLAB

1. Initial Comment

```
% Algoritmo para resolver la ecuación cuadrática ax^2+bx+c=0
```

- **Purpose:** Indicates that the code solves quadratic equations of the form $ax^2 + bx + c = 0$.
- **Note:** Comments in MATLAB start with % and are ignored during execution.

Solution Example 3 in MATLAB

2. Infinite Loop with while true

```
while true
```

- **Function:** Begins a loop that repeats indefinitely.
- **Purpose:** Allows the user to solve multiple equations consecutively without restarting the program.
- **Loop Exit:** The **break** command is used later to exit the loop when the user decides not to continue.

Solution Example 3 in MATLAB

3. Display Initial Message

```
disp('--- Resolución de ecuaciones cuadráticas ---')
```

- **Function disp:** Displays the given message on the screen, in this case, a title indicating the beginning of the quadratic equation solving process.

Solution Example 3 in MATLAB

4. Requesting Coefficients from the User

```
a = input('Ingrese el coeficiente a: ');
b = input('Ingrese el coeficiente b: ');
c = input('Ingrese el coeficiente c: ‘);
```

$$x_{1,2} = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

- **Function input:** Prompts the user to enter a value.
- **Purpose:** Collect the necessary coefficients to form the equation.

Solution Example 3 in MATLAB

5. Checking that a Is Not Zero

```
if a == 0
    disp('Error: El coeficiente "a" no puede ser cero en una ecuación cuadrática.');
else
```

- **Condition:** Checks if a is equal to zero.
- **Reason:** If a is zero, result in division by zero.
- **Action:**
  - If a is zero, an error message is displayed.
  - If a is not zero, the code proceeds to compute the equation.

Solution Example 3 in MATLAB

6. Calculating the Discriminant

```
discriminante = b^2 - 4*a*c;
```

Formula: The discriminant is calculated as $b^2 - 4ac$.

Discriminant

$$x_{1,2} = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

Importance:

- Determines the nature of the roots.
- If it is positive: two distinct real roots.
- If it is zero: two equal real roots.
- If it is negative: the roots are complex.

Solution Example 3 in MATLAB

7. Calculating the Square Root of the Discriminant

```
raiz_disc = sqrt(discriminante);
```

Function sqrt: Computes the square root of the discriminant.

Complex Number Handling: MATLAB automatically handles the conversion to complex numbers if the discriminant is negative, allowing imaginary roots to be obtained without error.

Solution Example 3 in MATLAB

8. Applying the Quadratic Formula

```
x1 = (-b + raiz_disc) / (2*a);
x2 = (-b - raiz_disc) / (2*a);
```

Variables:

- x1: The first solution (using the positive sign in the numerator).
- x2: The second solution (using the negative sign in the numerator).

Solution Example 3 in MATLAB

9. Displaying the Solutions

```
disp('Las soluciones de la ecuación son:')
fprintf('x1 = %f%+fi\n', real(x1), imag(x1));
fprintf('x2 = %f%+fi\n', real(x2), imag(x2));
```

disp: Prints a message informing the user that the solutions will be shown next.
fprintf: Prints the solutions in a formatted manner:

- %f is used to display the real part and %+fi displays the imaginary part, ensuring that the sign is shown.
- The functions real() and imag() extract the real and imaginary parts of each solution, which is useful if the roots are complex.
- The use of \n at the end of the format indicates a line break

Solution Example 3 in MATLAB

10. Asking the User if They Want to Solve Another Equation

```
repetir = input('¿Desea resolver otra ecuación? (s/n): ', 's');
```

- **User Input:** The user is prompted to indicate if they want to continue solving more equations.
- **Argument 's':** Specifies that the input should be treated as a string.

Solution Example 3 in MATLAB

11. Evaluating the User's Response

```
if lower(repetir) ~= 's'
    break;
end
```

lower(repetir): Converts the user's response to lowercase so that the comparison is case-insensitive.

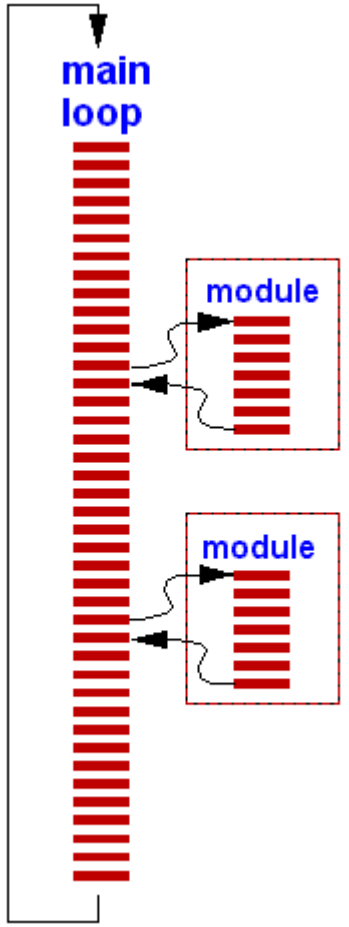
Condition:

- If the response is not 's', the break command is executed, which exits the while loop and ends the program.
- If it is 's', the loop continues.

Modular Programming

Concept and Advantages

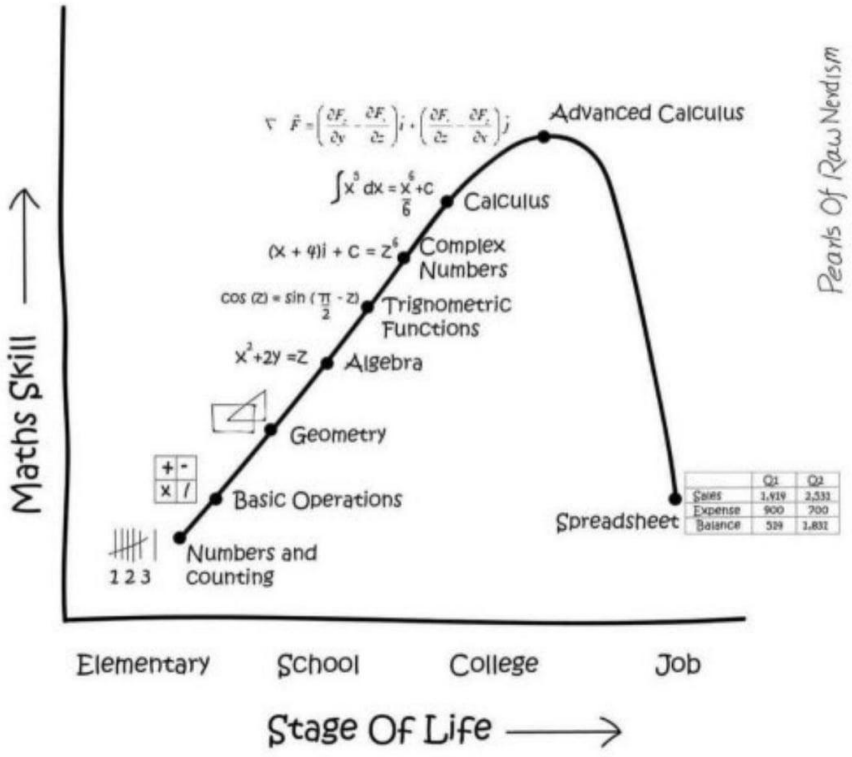
- Dividing a large program into small, self-contained modules (functions or subroutines)
- Each module performs a specific task and has one entry and one exit point
- Benefits include simplified debugging, testing in isolation, reusability, and easier maintenance



Excel as a Computational Environment

Spreadsheet Capabilities

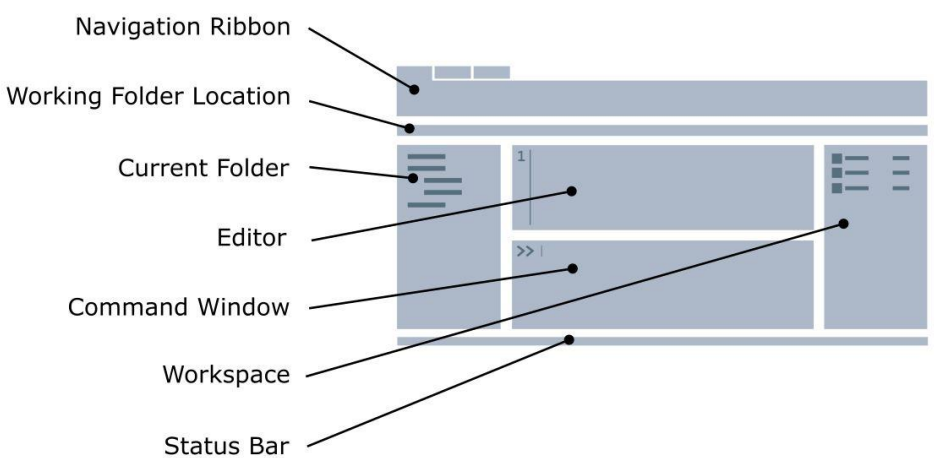
- Excel provides a grid format for entering and calculating data
- Ideal for “what if” analysis where changes in one cell automatically update dependent results
- Supports built-in numerical functions (e.g., exponentials, logarithms) that can be directly used in cells



MATLAB: A Numerical and Graphical Environment

Overview of MATLAB

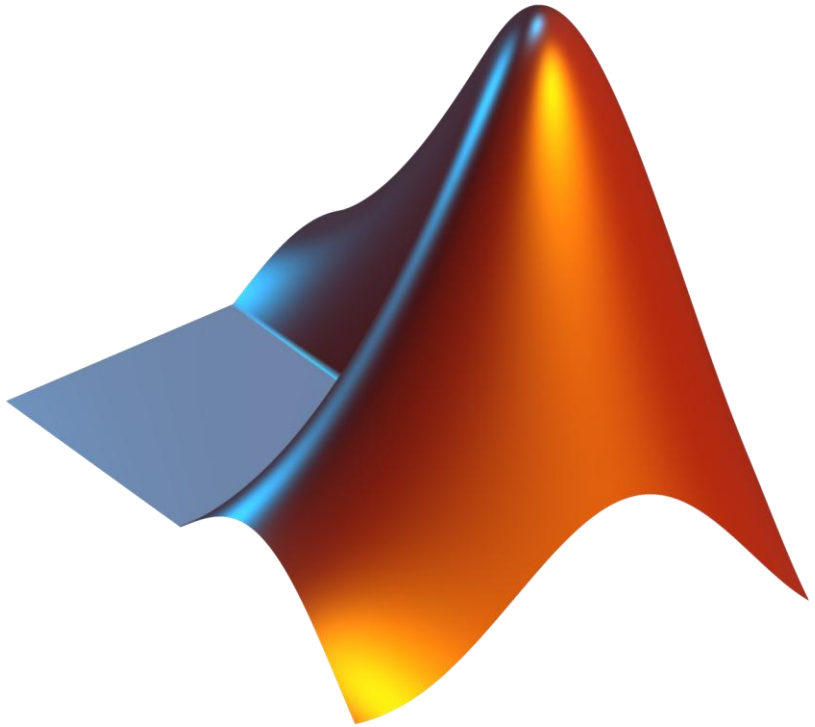
- Originally developed for matrix computations, MATLAB now provides an interactive platform for a wide range of numerical methods
- Combines a command window for immediate operations and an editor for creating scripts (M-files) and functions.



MATLAB: A Numerical and Graphical Environment

Programming in MATLAB

- Direct execution of arithmetic and matrix operations in the command window.
- Use of decision-making (if/else) and looping constructs (while loops, for loops) in M-files.



Python's Role

- Python has become one of the most popular languages in scientific computing.
- Extensive libraries like NumPy, SciPy, and matplotlib provide powerful tools for numerical analysis, visualization, and even symbolic computation (with packages such as SymPy).

5. Numerical Errors and Approximations

Introduction to Approximations and Round-Off Errors

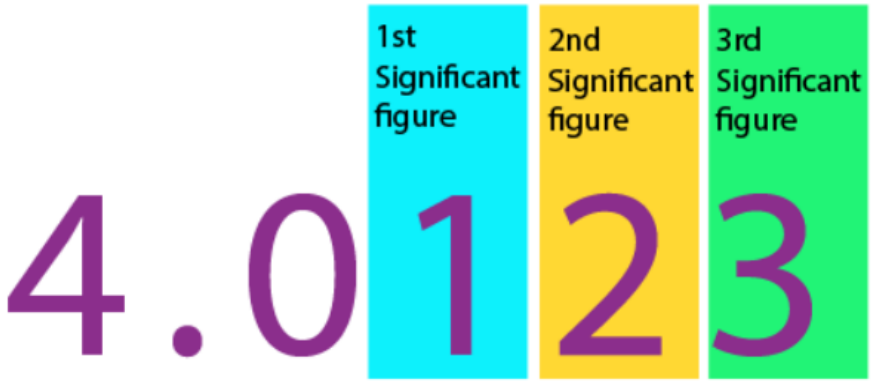
- It is important to understand numerical errors before applying numerical methods.
- When an analytical solution is not available, we estimate the error in numerical approximations.

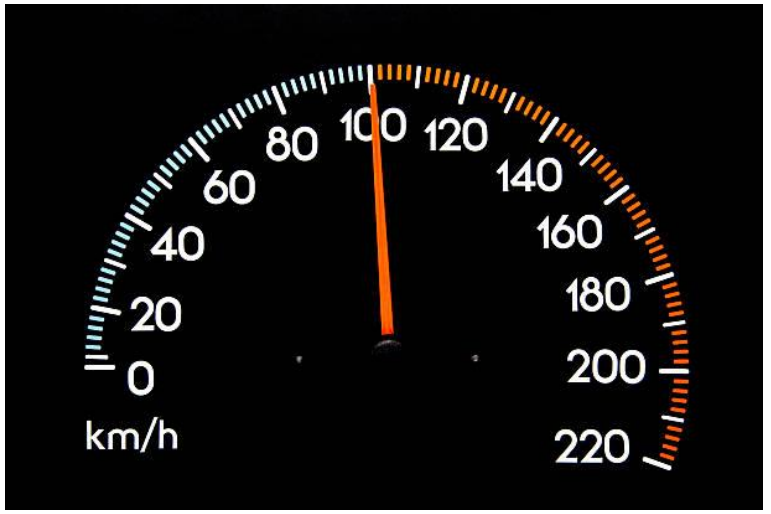
Significant Figures

Definition and Concept:

- Significant figures represent the digits that can be trusted in a measurement or computation (i.e., all the certain digits plus one estimated digit).

- Example: A speedometer reading might only provide two certain digits (e.g., 48 km/h) and one estimated digit (e.g., 48.5 km/h).

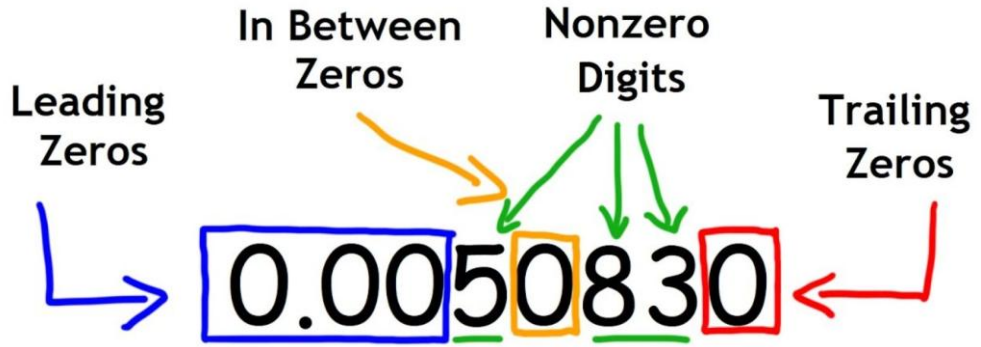




Significant Figures

Rules and Conventions:

- Zeros used to locate the decimal point are not necessarily significant.
- Trailing zeros in large numbers may be ambiguous; scientific notation (e.g., 4.53×10^4 vs. 4.530×10^4) resolves this by clearly indicating the number of significant figures.

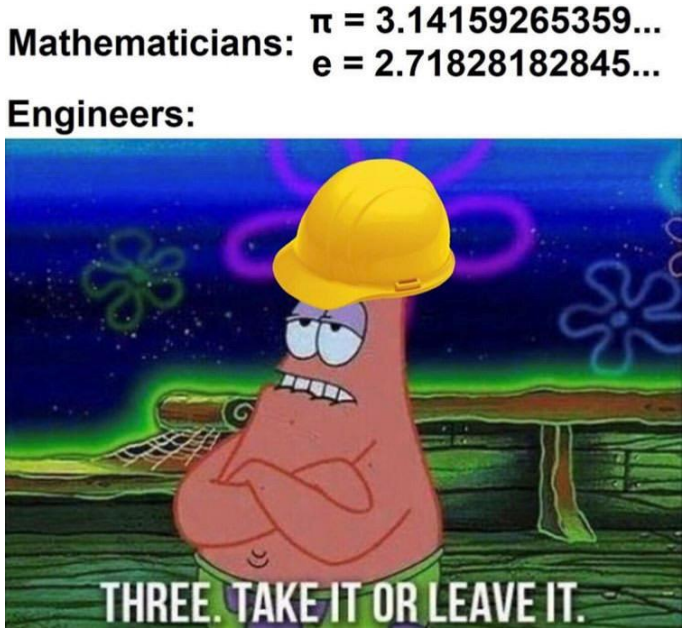


Significant Figures

Implications for Numerical Methods:

- Numerical approximations must be expressed in terms of significant figures to indicate **confidence in the result**.

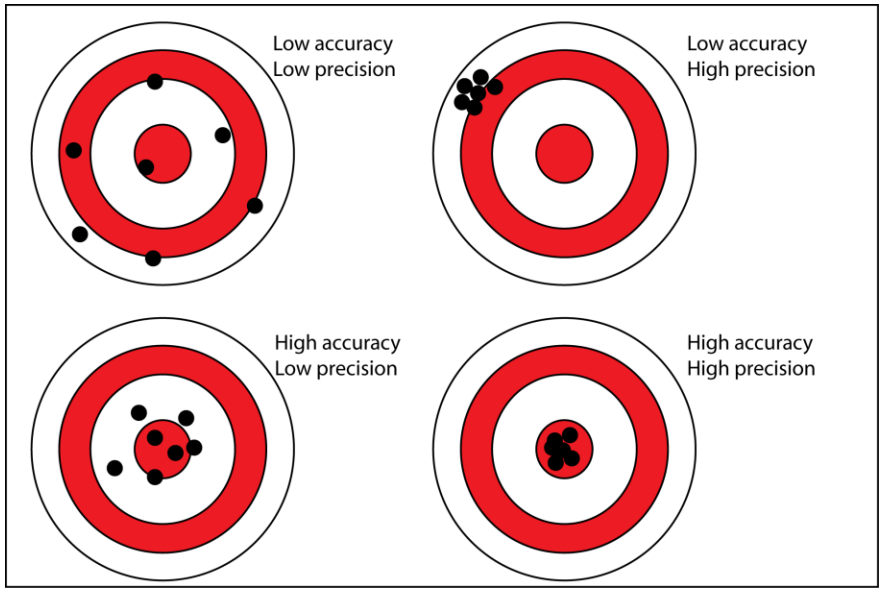
- The inherent limitation in representing numbers like **π** or **e** with a finite number of digits leads directly to round-off error.



Accuracy and Precision

Definitions:

- **Accuracy:** How closely a measured or computed value agrees with the true value.
- **Precision:** How consistently individual measurements or computed values agree with each other.



Error

The exact (or true) value of the error is:

$$E_t = \text{True Value} - \text{Approximation}$$

True Percent Relative Error:

$$\varepsilon_t = \frac{true\ error}{true\ value} \times 100\%$$

Example 4

Imagine you're tasked with determining the length of two objects: a large bridge and a small rivet. Your measurements result in lengths of 9,999 cm for the bridge and 9 cm for the rivet. If the actual lengths are 10,000 cm for the bridge and 10 cm for the rivet, calculate:

(a) the absolute true error, and

(b) the corresponding true percent relative error for each measurement.

Approximating Error in Iterative Methods:

In real-world applications, we do not know the true value.

When the true value is unknown, the error is estimated as the difference between successive approximations:

$$\varepsilon_a = \frac{Current\ Approximation - Previous\ Approximation}{Current\ Approximation} \times 100\%$$

Stopping Criterion: The iterative process continues until $|\varepsilon_a| < \varepsilon_s$ (where ε_s prespecified percent tolerance)

A relationship is established between the tolerance ε_s and the number of significant figures is $\varepsilon_s = 0.5 \times 10^{2-n}\%$ (Scarborough, 1966)

Example 5

In mathematics, many functions can be expressed as infinite series expansions. Consider, for example, the exponential function called **Maclaurin series expansion**, which can be expanded as:

$$e^x = 1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \cdots + \frac{x^n}{n!}$$

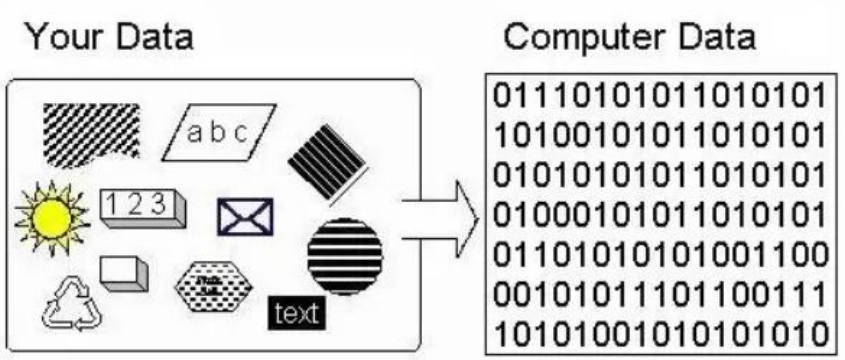
As additional terms are included, the approximation increasingly approaches the true value. Using this idea, begin with the simplest approximation for $e^{0.5}$, which is $e^{0.5} \approx 1$, and sequentially add subsequent terms from the series. After each term, calculate both the true and approximate percent relative errors. The exact value is known to be $e^{0.5} = 1{,}648721\ldots$ Continue adding terms until the approximate error is below a specified tolerance ε_s, represented with three significant figures.

Round-Off Errors

- Arise because **computers cannot store infinitely many digits**.

- **Key Reason**: Digital machines use fixed-bit representations (floating-point format), leading to small discrepancies in storage.

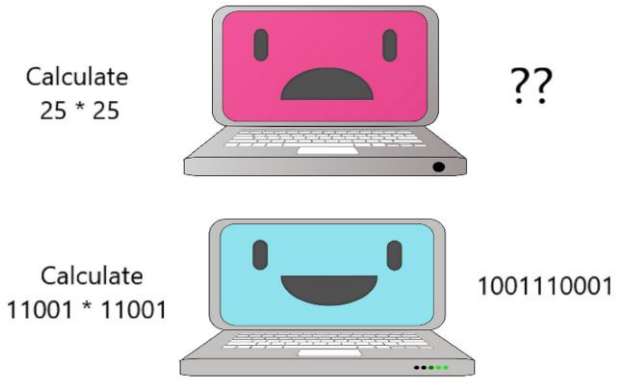
- **Finite Precision**:
  - Numbers like π or $\sqrt{2}$ cannot be represented exactly; they are approximated (rounded or chopped).

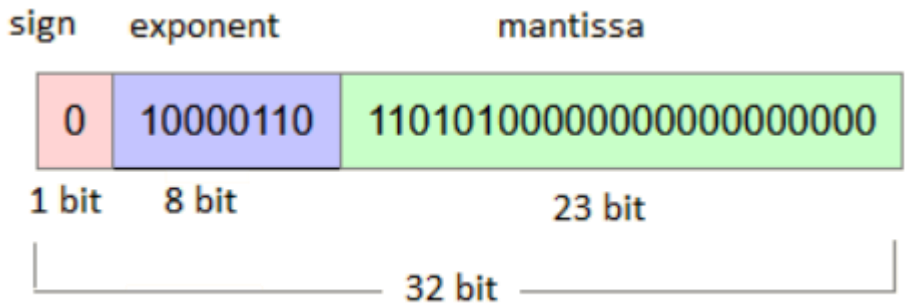


Computer Representation of Numbers

- **Integer Representation**: Limited range; overflow occurs if you exceed that range.

- **Floating-Point Representation**:
  - A number is stored with a *mantissa* (significant digits) and an *exponent* (scale).
  - Normalization ensures the mantissa has a fixed form.





Example of Computer Representation of Numbers

Base-10 Example (intuitive)

Consider the number

$$123.45$$

To normalize it (mantissa between 0.1 and 1.0):

$$123.45 = 0.12345 \times 10^3$$

- **Mantissa**: 0.12345
- **Exponent**: 3

Example of Computer Representation of Numbers

Binary & IEEE-754 Example (single precision)

Now take

$$5.755$$

Convert to binary (unnormalized):

$$5.75_{10} = 101.11_2$$

Normalize (mantissa between 1.0 and 2.0 in base 2):

$$101.11_2 = 1.0111_2 \times 2^2$$

- Mantissa (significand): 1.0111_2
- Exponent: 2

Example of Computer Representation of Numbers

IEEE-754 32-bit encoding:

- **Sign bit**: 0 (positive)
- **Exponent with bias 127**:

  $2+127=129=10000001_2$

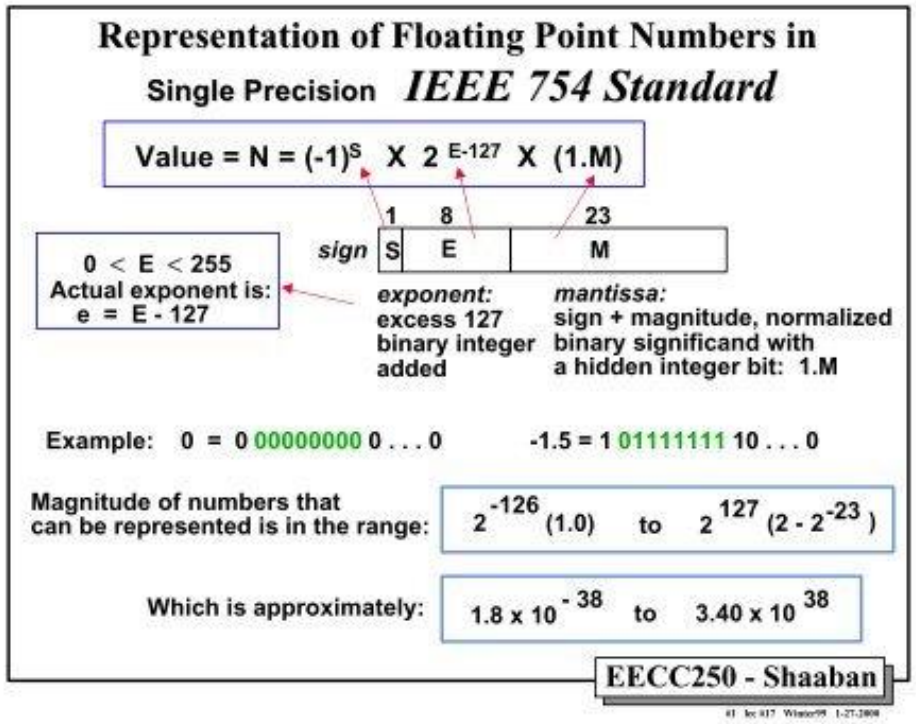
- **Mantissa field (omitting the implicit "1."):**

  01110000000000000000000

- **So the 32-bit pattern is:**

  [0] [10000001] [01110000000000000000000]

- Bit 0: sign
- Bits 1–8: exponent
- Bits 9–31: fraction (mantissa)



Arithmetic Manipulations

- **Addition/Subtraction**: When exponents differ greatly, adding a very small number to a large number can result in the small number being "lost."

- **Multiplication/Division**: Typically involve intermediate alignment or double-length registers to reduce round-off, but errors can still propagate.

- **Example**: Summing 0.00001 repeatedly can accumulate round-off error if 0.00001 is itself already an approximation in the machine's floating-point system.

6. Introduction to Truncation Errors and Taylor Series

Introduction

- **Definition**: Errors arising when an approximate procedure replaces an exact mathematical operation.

- **Illustrative example**: Finite-difference approximation of a derivative

$$\frac{dv}{dt} \approx \frac{v(t_{i+1}) - v(t_i)}{t_{i+1} - t_i}$$

  and the resulting discrepancy compared to the true derivative.

- **Importance**: Understanding truncation errors is key to designing accurate numerical methods.

Taylor's Theorem and the Remainder

The theorem states that any smooth function can be approximated as a polynomial. If f and its first n+1 derivatives are continuous on $[a,x]$, then

$$f(x) = f(a) + f'(a)(x-a) + \frac{f''(a)}{2!}(x-a)^2 + \cdots + \frac{f^n(a)}{n!}(x-a)^n + R_n$$

General nth-order expansion:

$$f(x_{i+1}) = \sum_{k=0}^{n} \frac{f^{(k)}(x_i)}{k!} h^k + R_n$$

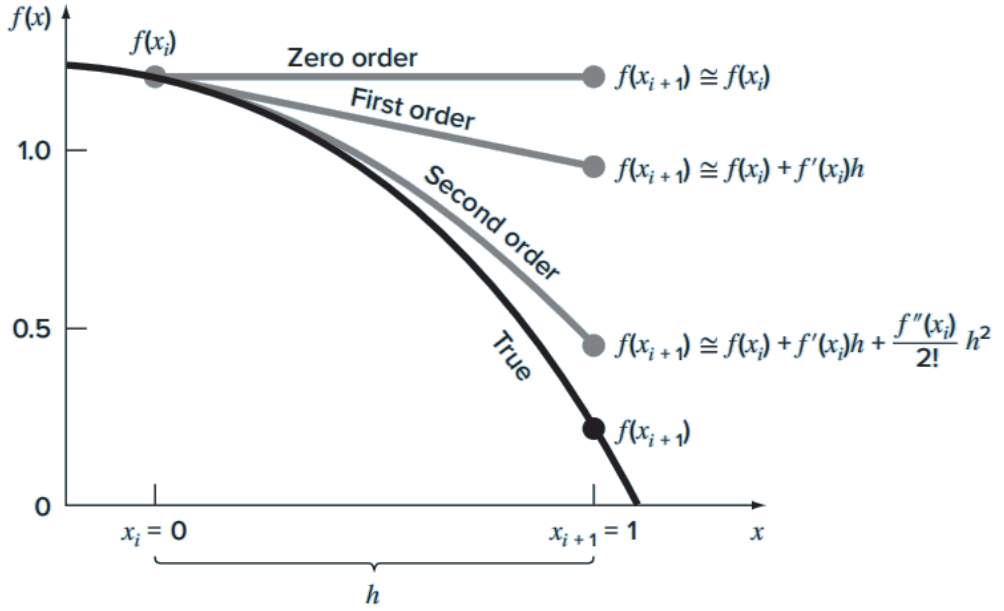
$$h = x_{i+1} - x_i$$

Constructing the Taylor Series Term by Term

Zero-order approximation:

$$f(x_{i+1}) \approx f(x_i)$$

Exact only if f is constant.



First-order (linear) approximation:

$$f(x_{i+1}) \approx f(x_i) + f'(x_i)(x_{i+1} - x_i)$$

Captures a straight-line trend.

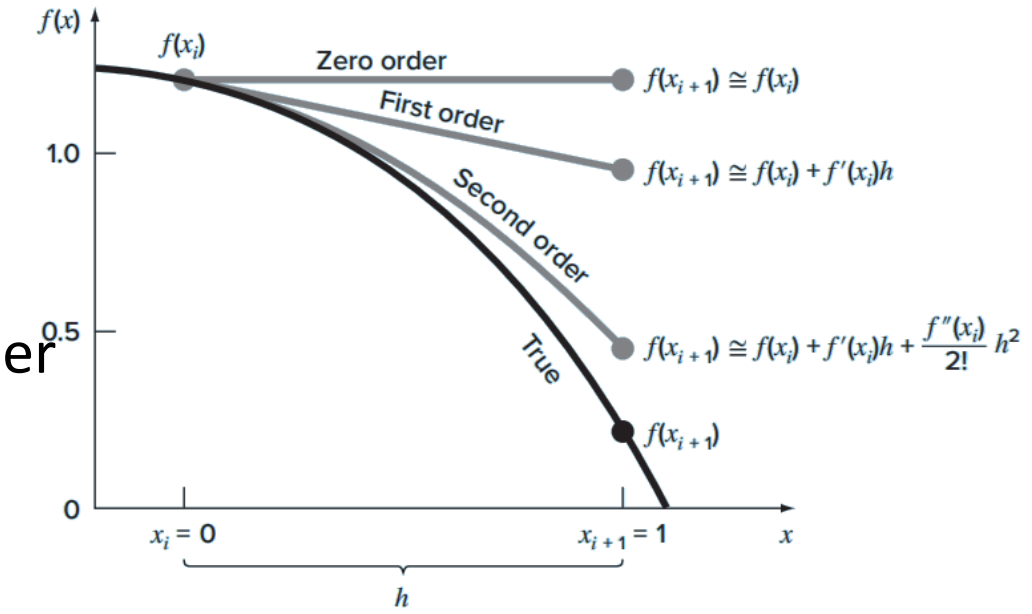
Second-order (quadratic) approximation:

$$f(x_{i+1}) \approx f(x_i) + f'(x_i)h + \frac{f''(x_i)}{2!}h^2, \qquad h = x_{i+1} - x_i$$

Example 6

For
$f(x) = -0.1x^4 - 0.15x^3 - 0.5x^2 - 0.25x + 1.2$

With $x_i = 0$ and $h = 1$, the zero- through fourth-order Taylor approximations at $x_{i+1} = 1$



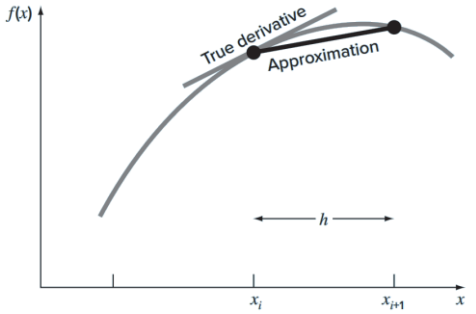
Example 7

Use Taylor series expansions with $n = 0$ to 6 to approximate $f(x) = \cos x$ at $x_{i+1} = \pi / 3$ on the basis of the value of $f(x)$ and its derivatives at $x_i = \pi / 4$. Note that this means that $h = \pi/3 - \pi / 4 = \pi / 12$.

Finite-Difference Formulas from Taylor Series

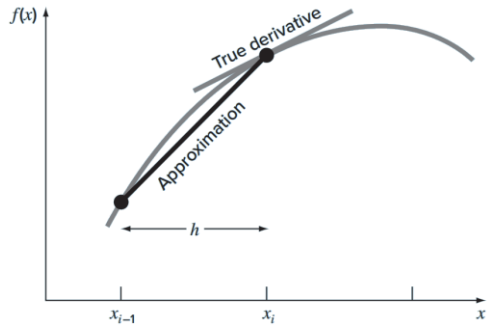
- **Forward Difference Approximation of the First Derivative:**

$$f'(x_i) = \frac{f(x_{i+1}) - f(x_i)}{h} + O(h)$$



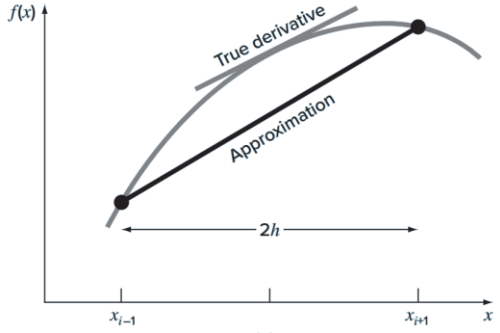
- **Backward Difference Approximation of the First Derivative :**

$$f'(x_i) = \frac{f(x_i) - f(x_{i-1})}{h} + O(h)$$



- **Center Difference Approximation of the First Derivative :**

$$f'(x_i) = \frac{f(x_{i+1}) - f(x_{i-1})}{2h} - O(h^2)$$



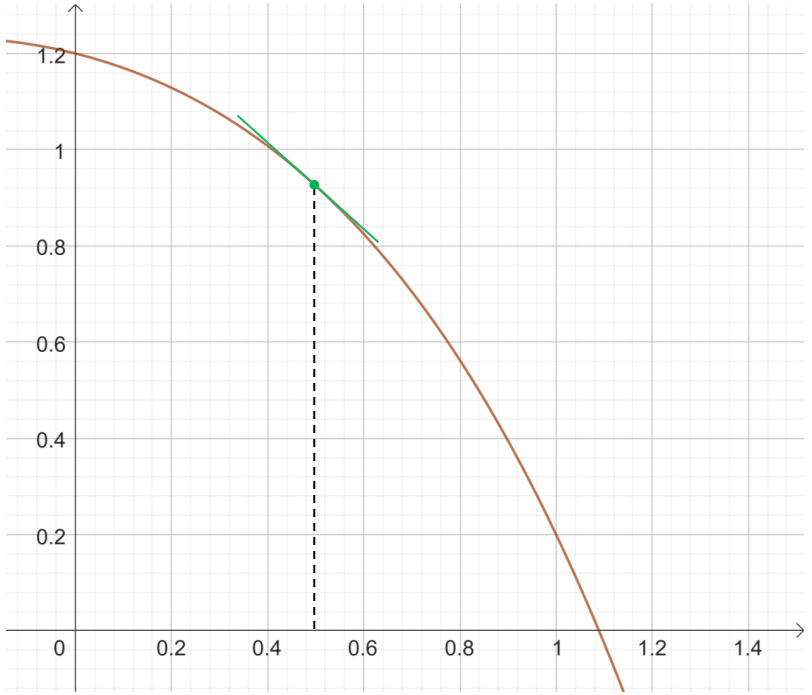
The nomenclature $O(h^{n+1})$ means that the truncation error is of the order of h^{n+1}.

Example 8

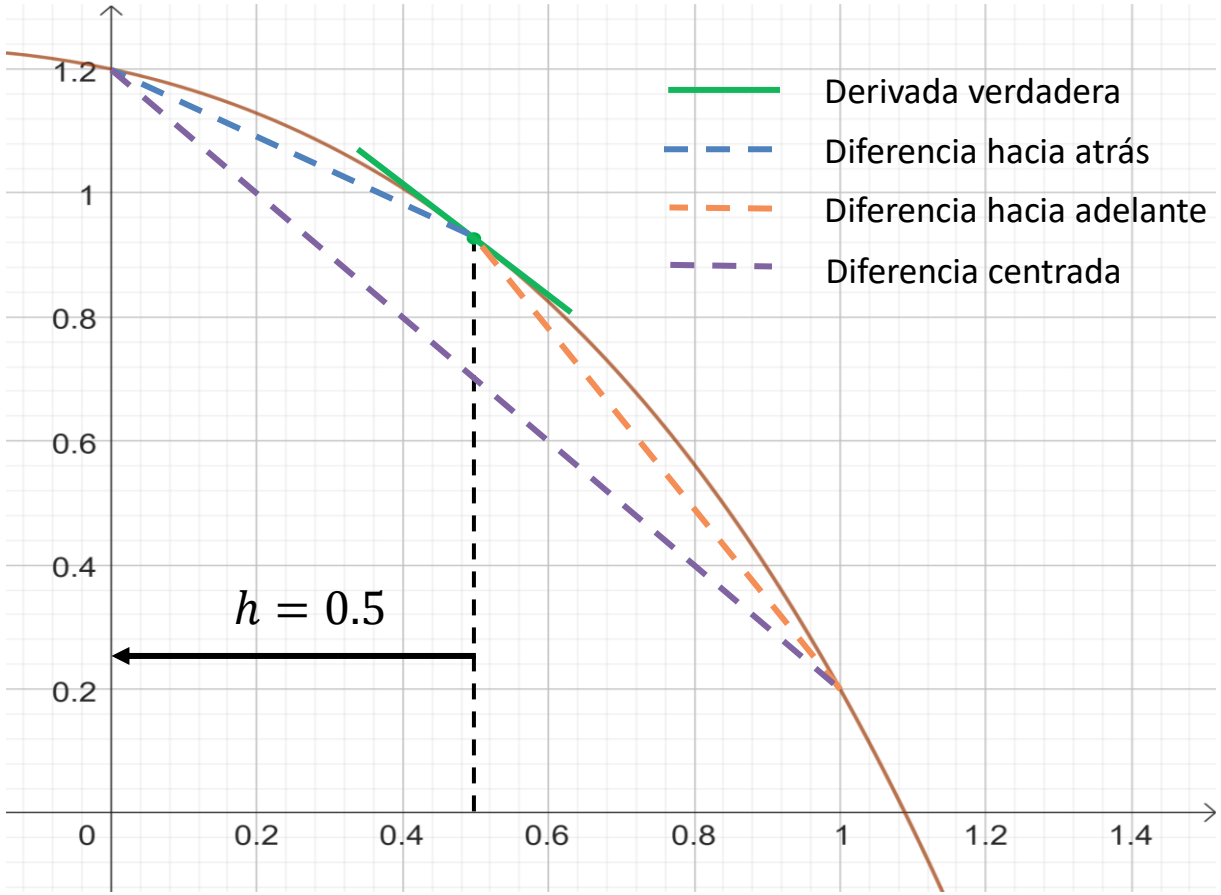
Use forward and backward difference approximations of O(h) and a centered difference approximation of O(h^2) to estimate **the first derivative** of

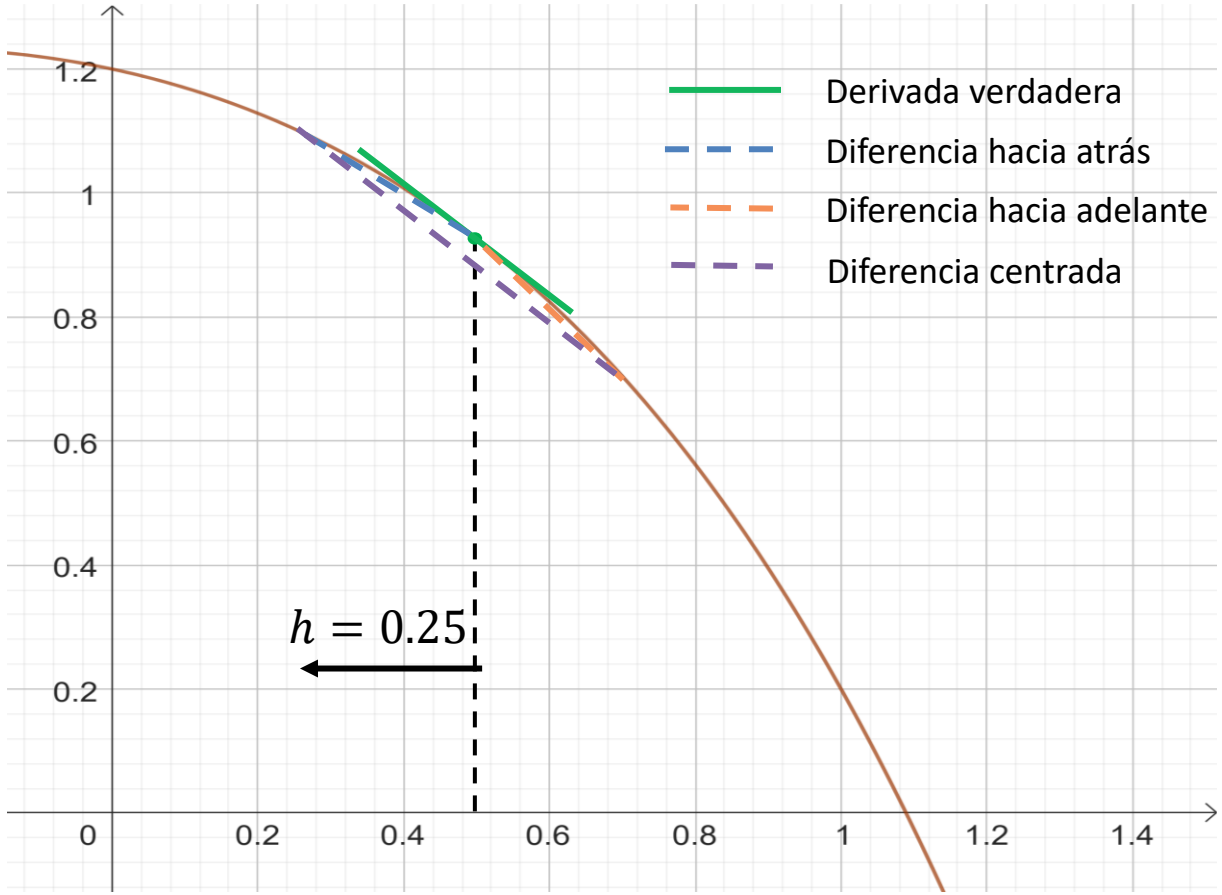
$$f(x) = -0.1x^4 - 0.15x^3 - 0.5x^2 - 0.25x + 1.2$$

at x = 0.5 using a step size of h = 0.5. Repeat the computation using h = 0.25



Example 8


Derivada verdadera
Diferencia hacia atrás
Diferencia hacia adelante
Diferencia centrada
$h = 0.5$
1.2
1
0.8
0.6
0.4
0.2
0
0.2
0.4
0.6
0.8
1
1.2
1.4


Derivada verdadera
Diferencia hacia atrás
Diferencia hacia adelante
Diferencia centrada
$h = 0.25$
1.2
1
0.8
0.6
0.4
0.2
0
0.2
0.4
0.6
0.8
1
1.2
1.4

Practical problem - Bonus

A cantilever of length L is subjected to a constant load w (force per unit length). The vertical displacement y at a point located a distance x from the clamped end can be found from the closed-form relation

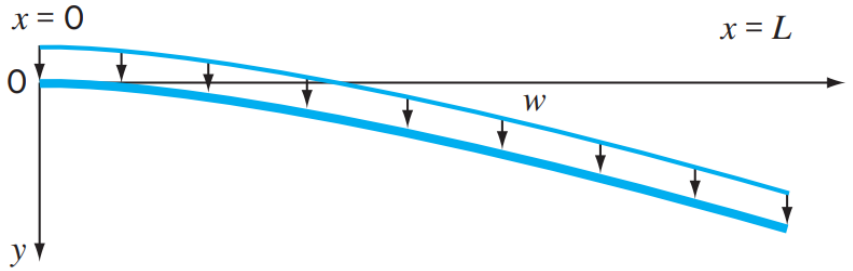
$$y(x) = \frac{w}{24EI}(x^4 - 4Lx^3 + 6L^2x^2)$$

Where: E is the elastic modulus ($2 \times 10^{11}\ Pa$), I is the second moment of area ($3.3 \times 10^{-4}\ m^4$), w is the uniform load intensity ($12000\ N/m$), L is the total beam length (4 m).

If one differentiates with respect to x, the slope of the deflection curve becomes

$$\frac{dy}{dx} = \frac{w}{24EI}(4x^3 - 12Lx^2 + 12L^2x)$$

Assuming the deflection at the support is zero, $y(0) = 0$, implement the forward-Euler Method with step size Δx = 0.125 m to march from x = 0 up to x = L. Compute the sequence of approximate deflections and then plot these numerical values together with the exact profile given by the formula above.



Summary

- **Definition of Numerical Methods:** Techniques that reformulate mathematical problems into sequences of arithmetic operations, leveraging digital computers for intensive, repetitive calculations otherwise infeasible analytically

- **Analytical vs. Numerical:** Analytical solutions provide insight but are limited to simple, often linear problems; numerical methods extend capabilities to complex geometries and nonlinear processes common in engineering

- **Historical Context:**
  - *Precomputer Era:* Reliance on calculators and slide rules focused efforts on computation rather than problem formulation and interpretation.
  - *Computer Era:* Fast computing shifts emphasis to comprehensive modeling and deep interpretation, enhancing overall problem-solving

Summary

- **Engineering Problem-Solving Phases:**
  1. **Formulation:** Relate real systems to fundamental laws.
  2. **Solution:** Apply numerical algorithms via computer methods.
  3. **Interpretation:** Analyze sensitivity, behavior, and system implications

- **Programming Foundations:** Differentiates between standard users (built-in functions) and power users (custom scripts); emphasizes structured programming, control structures (selection and loops), and modular design for maintainability and error reduction

Summary

Error Definitions

True error

$$E_t = \text{true value} - \text{approximation}$$

True percent relative error

$$\varepsilon_t = \frac{\text{true value} - \text{approximation}}{\text{true value}} 100\%$$

Approximate percent relative error

$$\varepsilon_a = \frac{\text{present approximation} - \text{previous approximation}}{\text{present approximation}} 100\%$$

Stopping criterion

Terminate computation when

$$\varepsilon_a < \varepsilon_s$$

where ε_s is the desired percent relative error

Summary

Numerical Errors & Approximations:

- **Round-off Errors:** Limitations from finite significant figures and floating-point representation.
- **Truncation Errors:** Arise from series approximations (e.g., Taylor series).
- **Accuracy vs. Precision:** Definitions and implications for engineering measurements

Introduction to Truncation Errors & Taylor Series: Construction of series, remainder term, finite-difference formulas, and practical examples illustrating error estimation in derivative approximations .